

Design Patterns: A Complete Guide

A structured and beginner-friendly reference to the three major categories of software design patterns — Creational, Structural, and Behavioral.

Design patterns are reusable solutions to commonly occurring problems in software design. They are not code — they are blueprints or templates that guide how to structure your code to solve a problem efficiently and elegantly.

Table of Contents

- [1. Creational Design Patterns](#)
 - [Singleton](#)
 - [Factory Method](#)
 - [Abstract Factory](#)
 - [Builder](#)
 - [Prototype](#)
 - [2. Structural Design Patterns](#)
 - [Adapter](#)
 - [Bridge](#)
 - [Composite](#)
 - [Decorator](#)
 - [Facade](#)
 - [Flyweight](#)
 - [Proxy](#)
 - [3. Behavioral Design Patterns](#)
 - [Chain of Responsibility](#)
 - [Command](#)
 - [Iterator](#)
 - [Mediator](#)
 - [Memento](#)
 - [Observer](#)
 - [State](#)
 - [Strategy](#)
 - [Template Method](#)
 - [Visitor](#)
-

1. Creational Design Patterns

Overview

Creational patterns deal with **how objects are created**. They abstract the instantiation process, making a system independent of how its objects are created, composed, and represented. These patterns give you more flexibility in deciding which objects need to be created for a given situation.

Singleton

Description: Ensures that a class has only **one instance** throughout the lifetime of an application and provides a global access point to that instance.

- **Why it is used:** To control access to shared resources such as a configuration manager, logger, or database connection — where having multiple instances would cause conflicts or waste resources.
 - **Where it is commonly used:** Logging systems, application configuration managers, thread pools, caching systems.
-

Code Example (DART)

```
/*
 * SINGLETON DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Singleton pattern ensures a class has only one instance and provides
a global point of access to that instance.
 *
 * WHY DOES IT EXIST?
 * - Some resources should only exist once (database connections, logging
services, configuration managers)
 * - Prevents multiple instances from causing conflicts or inconsistent
state
 * - Provides controlled access to the sole instance
 *
 * WHEN TO USE IT?
 * - When exactly one instance of a class is needed to coordinate actions
across the system
 * - When the sole instance should be extensible by subclassing, and
clients should be able to use an extended instance without modifying their
code
 * - Common use cases: database connection pools, logging services,
configuration managers, thread pools, caches
 *
 * PROS:
 * - Controlled access to the sole instance
 * - Reduced namespace pollution (compared to global variables)
 * - Can be subclassed
 * - Can be lazy-loaded (created only when first needed)
 *
 * CONS:
 * - Violates Single Responsibility Principle (handles both its business
logic and instance creation)
 * - Can hide dependencies (making code harder to test)
 * - Global state can lead to unexpected behavior in multi-threaded
environments
 * - Difficult to unit test due to global state
 *
 * REAL-WORLD ANALOGY:
 * Think of a government - there's only one president or prime minister at
```

```

a time. Everyone who needs to interact with the head of state goes through
the same single instance. You can't have two presidents simultaneously
making conflicting decisions.
*/

// Example 1: Basic Singleton using factory constructor
class DatabaseConnection {
  // Private static variable to hold the single instance
  static DatabaseConnection? _instance;

  // Private named constructor
  DatabaseConnection._internal() {
    print('DatabaseConnection: Initializing connection...');
    _isConnected = true;
  }

  // Factory constructor that returns the single instance
  factory DatabaseConnection() {
    _instance ??= DatabaseConnection._internal();
    return _instance!;
  }

  bool _isConnected = false;
  int _queryCount = 0;

  // Business logic methods
  void executeQuery(String query) {
    if (_isConnected) {
      _queryCount++;
      print('Executing query #$_queryCount: $query');
    } else {
      print('Error: Not connected to database');
    }
  }

  int get queryCount => _queryCount;

  bool get connectionStatus => _isConnected;
}

// Example 2: Singleton using static getter with lazy initialization
class Logger {
  static Logger? _instance;

  // Private constructor
  Logger._() {
    print('Logger: Initializing logger...');
  }

  // Static getter for lazy initialization
  static Logger get instance {
    _instance ??= Logger._();
    return _instance!;
  }
}

```

```

    int _logCount = 0;

    void log(String message) {
        _logCount++;
        print('[LOG #$_logCount] $message');
    }

    int get logCount => _logCount;
}

// Example 3: Singleton with initialization parameters
class ConfigurationManager {
    static ConfigurationManager? _instance;

    final Map<String, String> _config = {};

    // Private constructor with optional parameters
    ConfigurationManager._({Map<String, String>? initialConfig}) {
        print('ConfigurationManager: Initializing...');
        if (initialConfig != null) {
            _config.addAll(initialConfig);
        }
    }

    // Static method with optional initialization parameters
    static ConfigurationManager getInstance({Map<String, String>?
initialConfig}) {
        _instance ??= ConfigurationManager._(initialConfig: initialConfig);
        return _instance!;
    }

    void set(String key, String value) {
        _config[key] = value;
    }

    String? get(String key) {
        return _config[key];
    }

    void displayConfig() {
        print('Current Configuration:');
        _config.forEach((key, value) {
            print('  $key: $value');
        });
    }
}

// Example 4: Thread-safe (Synchronized) Singleton
// In Dart, everything runs in a single-threaded event loop, so basic
singletons are inherently thread-safe.
// However, when dealing with ASYNC initialization, multiple callers might
trigger initialization twice.
class AsyncSingleton {

```

```

static AsyncSingleton? _instance;
static bool _isInitializing = false;

AsyncSingleton._internal() {
  print('AsyncSingleton: Instance created');
}

static Future<AsyncSingleton> getInstance() async {
  // If instance exists, return it
  if (_instance != null) return _instance!;

  // If already initializing, wait and check again (serialized access)
  while (_isInitializing) {
    await Future.delayed(Duration(milliseconds: 10));
    if (_instance != null) return _instance!;
  }

  _isInitializing = true;
  try {
    print('AsyncSingleton: Starting heavy async initialization...');
    await Future.delayed(Duration(milliseconds: 100)); // Simulate delay
    _instance = AsyncSingleton._internal();
  } finally {
    _isInitializing = false;
  }

  return _instance!;
}

// Example 5: Bypassing Singleton
// In Dart, Singletons are scoped to the current ISOLATE.
// A common "bypass" is when developers expect a singleton to share state
// across the whole app,
// but it fails when multiple isolates are used (e.g. background workers).
class BypassableSingleton {
  static final BypassableSingleton _instance =
  BypassableSingleton._internal();
  BypassableSingleton._internal();
  factory BypassableSingleton() => _instance;
}

// Demo code
void main() {
  print('=== Singleton Pattern Demo ===\n');

  // Example 1: Basic Singleton
  print('--- DatabaseConnection Example ---');
  var db1 = DatabaseConnection();
  db1.executeQuery('SELECT * FROM users');

  var db2 = DatabaseConnection();
  db2.executeQuery('SELECT * FROM products');
}

```

```

print('Same instance? ${identical(db1, db2) ? 'Yes' : 'No'}');
print('Total queries: ${db1.queryCount}\n');

// Example 2: Singleton with getter
print('--- Logger Example ---');
Logger.instance.log('Application started');
Logger.instance.log('User logged in');
Logger.instance.log('Processing request');
print('Total logs: ${Logger.instance.logCount}\n');

// Example 3: Singleton with initialization parameters
print('--- ConfigurationManager Example ---');
var initialConfig = <String, String>{
  'apiUrl': 'https://api.example.com',
  'timeout': '5000'
};
var config = ConfigurationManager.getInstance(initialConfig:
initialConfig);
config.displayConfig();

config.set('apiKey', 'abc123');
config.displayConfig();

var config2 = ConfigurationManager.getInstance();
print('Same instance? ${identical(config, config2) ? 'Yes' : 'No'}\n');

// Example 4: Thread-safe (Async) Singleton
print('--- AsyncSingleton Example ---');
// Simulating multiple concurrent calls
Future.wait([
  AsyncSingleton.getInstance(),
  AsyncSingleton.getInstance(),
  AsyncSingleton.getInstance(),
]).then((instances) {
  print('All instances are same: ${instances.every((i) => identical(i,
instances[0]))}');
});

// Example 5: Bypassing Singleton (Conceptual)
print('--- Bypassing Singleton (Conceptual) ---');
print('In Dart, you can bypass singletons via:');
print('1. Multiple Isolates: Each isolate gets its own memory and its own
Singleton instance.');
```

(This is the most common "gotcha" in Flutter background tasks)';

```

  print('2. Reflection: Using "dart:mirrors" (not available in
Flutter/Web).');
  print('3. Dependency Injection: Swapping the singleton for a mock during
testing.');
```

Factory Method

Description: Defines an interface for creating an object, but **lets subclasses decide** which class to instantiate. The parent class defers object creation to its child classes.

- **Why it is used:** To promote loose coupling by eliminating the need to bind application-specific classes into your code. The creation logic is centralized and easy to extend.
- **Where it is commonly used:** UI frameworks (creating different buttons for different OS), plugin systems, document generators.

Code Example (DART)

```
/*
 * FACTORY METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Factory Method pattern defines an interface for creating an object
but lets subclasses decide which class to instantiate.
 * It delegates the instantiation logic to child classes.
 *
 * WHY DOES IT EXIST?
 * - Decouples the creation of objects from their usage
 * - Allows a class to defer instantiation to subclasses
 * - Provides a way to extend the creation logic without modifying existing
code (Open/Closed Principle)
 * - Eliminates the need to bind application-specific classes into your
code
 *
 * WHEN TO USE IT?
 * - When a class can't anticipate the class of objects it must create
 * - When a class wants its subclasses to specify the objects it creates
 * - When classes delegate responsibility to one of several helper
subclasses, and you want to localize the knowledge of which helper subclass
is the delegate
 * - Common use cases: document creation, UI components, payment
processing, logging frameworks
 *
 * PROS:
 * - Avoids tight coupling between the creator and the concrete products
 * - Follows Open/Closed Principle (easy to introduce new products without
changing existing code)
 * - Single Responsibility Principle (product creation code is in one
place)
 * - Can be combined with other patterns (e.g., Singleton, Prototype)
 *
 * CONS:
 * - Can make the code more complex by introducing many subclasses
 * - May require creating a parallel inheritance hierarchy for creators and
products
 * - Can lead to excessive use of inheritance
 *
 * REAL-WORLD ANALOGY:
 * Think of a car manufacturing plant. The plant (factory) knows how to
```

build cars, but it doesn't know what specific type of car to build until an order comes in. The plant has a method to build cars, but the specific car model (sedan, SUV, truck) is determined by the order or by specialized assembly lines (subclasses).

```
*/
```

```
// Product interface - defines the common interface for all products
```

```
abstract class Document {
```

```
    void open();
```

```
    void close();
```

```
    void save();
```

```
    String get type;
```

```
}
```

```
// Concrete Product 1: Text Document
```

```
class TextDocument extends Document {
```

```
    @override
```

```
    void open() {
```

```
        print('Opening Text Document in text editor...');
```

```
    }
```

```
    @override
```

```
    void close() {
```

```
        print('Closing Text Document...');
```

```
    }
```

```
    @override
```

```
    void save() {
```

```
        print('Saving Text Document as .txt file...');
```

```
    }
```

```
    @override
```

```
    String get type => 'Text Document';
```

```
}
```

```
// Concrete Product 2: PDF Document
```

```
class PDFDocument extends Document {
```

```
    @override
```

```
    void open() {
```

```
        print('Opening PDF Document in PDF viewer...');
```

```
    }
```

```
    @override
```

```
    void close() {
```

```
        print('Closing PDF Document...');
```

```
    }
```

```
    @override
```

```
    void save() {
```

```
        print('Saving PDF Document as .pdf file...');
```

```
    }
```

```
    @override
```

```
    String get type => 'PDF Document';
```

```
}

// Concrete Product 3: HTML Document
class HtmlDocument extends Document {
  @override
  void open() {
    print('Opening HTML Document in web browser...');
  }

  @override
  void close() {
    print('Closing HTML Document...');
  }

  @override
  void save() {
    print('Saving HTML Document as .html file...');
  }

  @override
  String get type => 'HTML Document';
}

// Creator abstract class - declares the factory method
abstract class DocumentApplication {
  // Factory method - to be implemented by subclasses
  // This is the core of the pattern: subclasses decide what to create
  Document createDocument();

  // Business logic that uses the product
  // This method doesn't need to know which specific document is created
  void newDocument() {
    var doc = createDocument();
    print('Created: ${doc.type}');
    doc.open();
    doc.save();
    doc.close();
  }
}

// Concrete Creator 1: Text Editor Application
class TextEditor extends DocumentApplication {
  @override
  Document createDocument() {
    return TextDocument();
  }
}

// Concrete Creator 2: PDF Viewer Application
class PDFViewer extends DocumentApplication {
  @override
  Document createDocument() {
    return PDFDocument();
  }
}
```

```
}

// Concrete Creator 3: Web Browser Application
class WebBrowser extends DocumentApplication {
  @override
  Document createDocument() {
    return HtmlDocument();
  }
}

// Alternative implementation: Parameterized Factory Method
// This allows a single creator to create multiple types of products
enum DocType { text, pdf, html }

class DocumentFactory {
  static Document createDocument(DocType type) {
    switch (type) {
      case DocType.text:
        return TextDocument();
      case DocType.pdf:
        return PDFDocument();
      case DocType.html:
        return HtmlDocument();
    }
  }
}

// Example: Payment Processing Factory
abstract class PaymentProcessor {
  void processPayment(int amount);
  void refundPayment(int amount);
}

class CreditCardProcessor extends PaymentProcessor {
  @override
  void processPayment(int amount) {
    print('Processing credit card payment of \$$amount');
  }

  @override
  void refundPayment(int amount) {
    print('Refunding credit card payment of \$$amount');
  }
}

class PayPalProcessor extends PaymentProcessor {
  @override
  void processPayment(int amount) {
    print('Processing PayPal payment of \$$amount');
  }

  @override
  void refundPayment(int amount) {
    print('Refunding PayPal payment of \$$amount');
  }
}
```

```
}  
}  
  
class CryptoProcessor extends PaymentProcessor {  
  @override  
  void processPayment(int amount) {  
    print('Processing cryptocurrency payment of \$$amount');  
  }  
  
  @override  
  void refundPayment(int amount) {  
    print('Refunding cryptocurrency payment of \$$amount');  
  }  
}  
  
enum PaymentMethod { creditCard, payPal, crypto }  
  
class PaymentFactory {  
  static PaymentProcessor createProcessor(PaymentMethod method) {  
    switch (method) {  
      case PaymentMethod.creditCard:  
        return CreditCardProcessor();  
      case PaymentMethod.payPal:  
        return PayPalProcessor();  
      case PaymentMethod.crypto:  
        return CryptoProcessor();  
    }  
  }  
}  
  
// Demo code  
void main() {  
  print('=== Factory Method Pattern Demo ===\n');  
  
  // Using inheritance-based Factory Method  
  print('--- Inheritance-based Factory Method ---');  
  
  var textEditor = TextEditor();  
  textEditor.newDocument();  
  print('');  
  
  var pdfViewer = PDFViewer();  
  pdfViewer.newDocument();  
  print('');  
  
  var webBrowser = WebBrowser();  
  webBrowser.newDocument();  
  print('');  
  
  // Using parameterized Factory Method  
  print('--- Parameterized Factory Method ---');  
  
  var textDoc = DocumentFactory.createDocument(DocType.text);  
  textDoc.open();  
}
```

```

print('');

var pdfDoc = DocumentFactory.createDocument(DocType.pdf);
pdfDoc.open();
print('');

var htmlDoc = DocumentFactory.createDocument(DocType.html);
htmlDoc.open();
print('');

// Payment Processing Example
print('--- Payment Processing Example ---');

var creditCardProcessor =
PaymentFactory.createProcessor(PaymentMethod.creditCard);
creditCardProcessor.processPayment(100);
creditCardProcessor.refundPayment(50);
print('');

var paypalProcessor =
PaymentFactory.createProcessor(PaymentMethod.payPal);
paypalProcessor.processPayment(75);
print('');

var cryptoProcessor =
PaymentFactory.createProcessor(PaymentMethod.crypto);
cryptoProcessor.processPayment(200);
}

```

Abstract Factory

Description: Provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

- **Why it is used:** To ensure that a set of related objects are used together without mixing incompatible combinations. It's like a factory of factories.
- **Where it is commonly used:** Cross-platform UI toolkits (e.g., Windows vs. macOS components), database drivers, theme systems.

Code Example (DART)

```

/*
 * ABSTRACT FACTORY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Abstract Factory pattern provides an interface for creating families
 * of related or dependent objects without specifying their concrete classes.
 * It's like a factory of factories.
 *
 * WHY DOES IT EXIST?

```

```

* - When you need to create families of related objects (e.g., UI
components for different operating systems)
* - When you want to ensure that created objects are compatible with each
other
* - When you want to provide a library of products without exposing their
implementation details
* - When you need to support multiple themes or platforms
*
* WHEN TO USE IT?
* - When a system should be independent of how its products are created,
composed, and represented
* - When a system should be configured with one of multiple families of
products
* - When the family of related product objects is designed to be used
together, and you need to enforce this constraint
* - When you want to provide a class library of products, and you want to
reveal just their interfaces, not their implementations
* - Common use cases: cross-platform UI components, database connectors
for different DBMS, game development (different themes)
*
* PROS:
* - Ensures that products from the same family are compatible
* - Hides the concrete classes from the client code
* - Easy to introduce new product families without modifying existing code
(Open/Closed Principle)
* - Promotes consistency among products
*
* CONS:
* - Can be complex to implement due to many interfaces and classes
* - Adding new products to existing families requires modifying all
factory interfaces
* - Can lead to excessive abstraction and indirection
* - May be overkill for simple scenarios
*
* REAL-WORLD ANALOGY:
* Think of a furniture factory that produces different styles of furniture
(Modern, Victorian, Rustic). Each style has its own family of products
(chairs, tables, sofas). If you order a Modern chair, you want it to match
with a Modern table. The abstract factory ensures that all furniture pieces
you order belong to the same style family.
*/

// Abstract Product A: Button interface
abstract class Button {
  void click();
  void render();
  String get name;
}

// Abstract Product B: Checkbox interface
abstract class Checkbox {
  void check();
  void render();
  String get name;
}

```

```
}

// Concrete Product A1: Windows Button
class WindowsButton extends Button {
  @override
  void click() {
    print('Windows button clicked!');
  }

  @override
  void render() {
    print('Rendering Windows-style button...');
  }

  @override
  String get name => 'Windows Button';
}

// Concrete Product B1: Windows Checkbox
class WindowsCheckbox extends Checkbox {
  @override
  void check() {
    print('Windows checkbox checked!');
  }

  @override
  void render() {
    print('Rendering Windows-style checkbox...');
  }

  @override
  String get name => 'Windows Checkbox';
}

// Concrete Product A2: Mac Button
class MacButton extends Button {
  @override
  void click() {
    print('Mac button clicked!');
  }

  @override
  void render() {
    print('Rendering Mac-style button...');
  }

  @override
  String get name => 'Mac Button';
}

// Concrete Product B2: Mac Checkbox
class MacCheckbox extends Checkbox {
  @override
  void check() {
```

```
        print('Mac checkbox checked!');
    }

    @override
    void render() {
        print('Rendering Mac-style checkbox...');
    }

    @override
    String get name => 'Mac Checkbox';
}

// Abstract Factory interface - declares methods for creating each abstract
// product
abstract class GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Concrete Factory 1: Windows Factory
// Creates Windows-specific UI components
class WindowsFactory extends GUIFactory {
    @override
    Button createButton() {
        return WindowsButton();
    }

    @override
    Checkbox createCheckbox() {
        return WindowsCheckbox();
    }
}

// Concrete Factory 2: Mac Factory
// Creates Mac-specific UI components
class MacFactory extends GUIFactory {
    @override
    Button createButton() {
        return MacButton();
    }

    @override
    Checkbox createCheckbox() {
        return MacCheckbox();
    }
}

// Client class that uses the abstract factory
// The client doesn't know which concrete factory is being used
class GUIApplication {
    final Button _button;
    final Checkbox _checkbox;

    GUIApplication(GUIFactory factory)
```

```
        : _button = factory.createButton(),
          _checkbox = factory.createCheckbox();

// Business logic that uses the products
void renderUI() {
    print('Rendering UI components...');
    print('Button: ${_button.name}');
    print('Checkbox: ${_checkbox.name}');

    _button.render();
    _checkbox.render();
}

void interact() {
    print('\nUser interaction...');
    _button.click();
    _checkbox.check();
}
}

// Example: Database Abstract Factory
// Shows how Abstract Factory can be used for database connections

// Abstract Product: Database Connection
abstract class IDatabaseConnection {
    void connect();
    void disconnect();
    void executeQuery(String query);
}

// Abstract Product: Database Command
abstract class DatabaseCommand {
    void execute();
}

// Concrete Products: MySQL
class MySQLConnection extends IDatabaseConnection {
    @override
    void connect() {
        print('Connecting to MySQL database...');
    }

    @override
    void disconnect() {
        print('Disconnecting from MySQL database...');
    }

    @override
    void executeQuery(String query) {
        print('Executing MySQL query: $query');
    }
}

class MySQLCommand extends DatabaseCommand {
```

```
    final String _command;

    MySQLCommand(this._command);

    @override
    void execute() {
        print('Executing MySQL command: $_command');
    }
}

// Concrete Products: PostgreSQL
class PostgreSQLConnection extends IDatabaseConnection {
    @override
    void connect() {
        print('Connecting to PostgreSQL database...');
    }

    @override
    void disconnect() {
        print('Disconnecting from PostgreSQL database...');
    }

    @override
    void executeQuery(String query) {
        print('Executing PostgreSQL query: $query');
    }
}

class PostgreSQLCommand extends DatabaseCommand {
    final String _command;

    PostgreSQLCommand(this._command);

    @override
    void execute() {
        print('Executing PostgreSQL command: $_command');
    }
}

// Abstract Factory for Database
abstract class DatabaseFactory {
    IDatabaseConnection createConnection();
    DatabaseCommand createCommand(String cmd);
}

// Concrete Factory: MySQL
class MySQLFactory extends DatabaseFactory {
    @override
    IDatabaseConnection createConnection() {
        return MySQLConnection();
    }

    @override
    DatabaseCommand createCommand(String cmd) {
```

```

        return MySQLCommand(cmd);
    }
}

// Concrete Factory: PostgreSQL
class PostgreSQLFactory extends DatabaseFactory {
    @override
    IDatabaseConnection createConnection() {
        return PostgreSQLConnection();
    }

    @override
    DatabaseCommand createCommand(String cmd) {
        return PostgreSQLCommand(cmd);
    }
}

// Demo code
void main() {
    print('=== Abstract Factory Pattern Demo ===\n');

    // Example 1: GUI Components
    print('--- GUI Components Example ---');

    // Create Windows UI
    print('\nCreating Windows UI:');
    var windowsFactory = WindowsFactory();
    var windowsApp = GUIApplication(windowsFactory);
    windowsApp.renderUI();
    windowsApp.interact();

    // Create Mac UI
    print('\nCreating Mac UI:');
    var macFactory = MacFactory();
    var macApp = GUIApplication(macFactory);
    macApp.renderUI();
    macApp.interact();

    // Example 2: Database Connections
    print('\n\n--- Database Connections Example ---');

    // MySQL
    print('\nUsing MySQL:');
    var mysqlFactory = MySQLFactory();
    var mysqlConnection = mysqlFactory.createConnection();
    var mysqlCommand = mysqlFactory.createCommand('CREATE TABLE users (id
INT)');

    mysqlConnection.connect();
    mysqlConnection.executeQuery('SELECT * FROM users');
    mysqlCommand.execute();
    mysqlConnection.disconnect();

    // PostgreSQL

```

```

print('\nUsing PostgreSQL:');
var postgresFactory = PostgreSQLFactory();
var postgresConnection = postgresFactory.createConnection();
var postgresCommand = postgresFactory.createCommand('CREATE TABLE
products (id INT)');

postgresConnection.connect();
postgresConnection.executeQuery('SELECT * FROM products');
postgresCommand.execute();
postgresConnection.disconnect();
}

```

Builder

Description: Separates the **construction of a complex object** from its representation, allowing the same construction process to create different representations.

- **Why it is used:** To construct objects step-by-step, especially when an object requires many configuration options or when different representations of a product are needed.
- **Where it is commonly used:** Building complex query objects, constructing HTML/XML documents, configuring network requests.

Code Example (DART)

```

/*
 * BUILDER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Builder pattern separates the construction of a complex object from
its representation, allowing the same construction process to create
different representations.
 * It constructs complex objects step by step.
 *
 * WHY DOES IT EXIST?
 * - When you need to create complex objects with many optional parameters
 * - When you want to separate the construction logic from the object's
representation
 * - When you need to create objects in a step-by-step fashion
 * - When you want to provide different representations for the same
construction process
 *
 * WHEN TO USE IT?
 * - When the algorithm for creating a complex object should be independent
of the parts that make up the object and how they're assembled
 * - When the construction process must allow different representations for
the object that's constructed
 * - When you want to construct objects with many optional parameters
without using a telescoping constructor
 * - Common use cases: building complex documents, SQL query builders,
HTML/XML generators, configuration objects

```

```

*
* PROS:
* - Allows you to construct complex objects step by step
* - Provides fine-grained control over the construction process
* - Can reuse the same construction process to create different
representations
* - Single Responsibility Principle (complex construction code is isolated
from business logic)
* - More readable than telescoping constructors
*
* CONS:
* - Increases overall code complexity
* - Requires creating a separate Builder class for each product
* - May not be necessary for simple objects
* - Can lead to code duplication if not carefully designed
*
* REAL-WORLD ANALOGY:
* Think of ordering a custom pizza. You don't just say "give me a pizza" -
you specify the crust, sauce, cheese, and toppings step by step. The pizza
builder (chef) takes your specifications and constructs the pizza. You can
use the same process to create different types of pizzas (vegetarian, meat
lover's, etc.) by choosing different options at each step.
*/

// Product class - the complex object being built
class House {
  String _foundation = '';
  String _structure = '';
  String _roof = '';
  String _interior = '';
  bool _hasGarage = false;
  bool _hasGarden = false;
  bool _hasSwimmingPool = false;

  // Setter methods for builder to use
  void setFoundation(String f) {
    _foundation = f;
  }

  void setStructure(String s) {
    _structure = s;
  }

  void setRoof(String r) {
    _roof = r;
  }

  void setInterior(String i) {
    _interior = i;
  }

  void setGarage(bool g) {
    _hasGarage = g;
  }
}

```

```
void setGarden(bool g) {
  _hasGarden = g;
}

void setSwimmingPool(bool s) {
  _hasSwimmingPool = s;
}

// Display the house details
void display() {
  print('=== House Details ===');
  print('Foundation: $_foundation');
  print('Structure: $_structure');
  print('Roof: $_roof');
  print('Interior: $_interior');
  print('Garage: ${_hasGarage ? 'Yes' : 'No'}');
  print('Garden: ${_hasGarden ? 'Yes' : 'No'}');
  print('Swimming Pool: ${_hasSwimmingPool ? 'Yes' : 'No'}');
  print('=====');
}
}

// Builder interface - defines the steps to build the product
abstract class HouseBuilder {
  void buildFoundation();
  void buildStructure();
  void buildRoof();
  void buildInterior();
  void buildGarage();
  void buildGarden();
  void buildSwimmingPool();
  House getResult();
}

// Concrete Builder 1: Simple House Builder
class SimpleHouseBuilder extends HouseBuilder {
  final House _house = House();

  @override
  void buildFoundation() {
    _house.setFoundation('Concrete foundation');
  }

  @override
  void buildStructure() {
    _house.setStructure('Wood frame structure');
  }

  @override
  void buildRoof() {
    _house.setRoof('Asphalt shingle roof');
  }
}
```

```
@override
void buildInterior() {
  _house.setInterior('Basic interior with drywall');
}

@override
void buildGarage() {
  _house.setGarage(false);
}

@override
void buildGarden() {
  _house.setGarden(false);
}

@override
void buildSwimmingPool() {
  _house.setSwimmingPool(false);
}

@override
House getResult() {
  return _house;
}
}

// Concrete Builder 2: Luxury House Builder
class LuxuryHouseBuilder extends HouseBuilder {
  final House _house = House();

  @override
  void buildFoundation() {
    _house.setFoundation('Reinforced concrete foundation with basement');
  }

  @override
  void buildStructure() {
    _house.setStructure('Steel frame structure with brick exterior');
  }

  @override
  void buildRoof() {
    _house.setRoof('Spanish tile roof with solar panels');
  }

  @override
  void buildInterior() {
    _house.setInterior('Luxury interior with marble floors and custom
cabinetry');
  }

  @override
  void buildGarage() {
    _house.setGarage(true);
  }
}
```

```
}

@override
void buildGarden() {
  _house.setGarden(true);
}

@override
void buildSwimmingPool() {
  _house.setSwimmingPool(true);
}

@override
House getResult() {
  return _house;
}
}

// Director class - orchestrates the construction process
// The director knows the construction algorithm but doesn't know the
// details
class ConstructionDirector {
  HouseBuilder? _builder;

  void setBuilder(HouseBuilder builder) {
    _builder = builder;
  }

  // Construct a basic house
  void buildBasicHouse() {
    _builder!.buildFoundation();
    _builder!.buildStructure();
    _builder!.buildRoof();
    _builder!.buildInterior();
  }

  // Construct a full-featured house
  void buildFullHouse() {
    _builder!.buildFoundation();
    _builder!.buildStructure();
    _builder!.buildRoof();
    _builder!.buildInterior();
    _builder!.buildGarage();
    _builder!.buildGarden();
    _builder!.buildSwimmingPool();
  }

  // Construct a custom house
  void buildCustomHouse(bool withGarage, bool withGarden, bool withPool) {
    _builder!.buildFoundation();
    _builder!.buildStructure();
    _builder!.buildRoof();
    _builder!.buildInterior();
    if (withGarage) _builder!.buildGarage();
  }
}
```

```
        if (withGarden) _builder!.buildGarden();
        if (withPool) _builder!.buildSwimmingPool();
    }
}

// Alternative: Fluent Builder pattern (method chaining)
// This is a common variation where the builder returns itself for chaining
class Computer {
    String _cpu = '';
    String _gpu = '';
    int _ram = 8;
    int _storage = 256;
    bool _hasWifi = false;
    bool _hasBluetooth = false;

    void setCpu(String cpu) {
        _cpu = cpu;
    }

    void setGpu(String gpu) {
        _gpu = gpu;
    }

    void setRam(int ram) {
        _ram = ram;
    }

    void setStorage(int storage) {
        _storage = storage;
    }

    void setWifi(bool wifi) {
        _hasWifi = wifi;
    }

    void setBluetooth(bool bluetooth) {
        _hasBluetooth = bluetooth;
    }

    void display() {
        print('=== Computer Specs ===');
        print('CPU: $_cpu');
        print('GPU: $_gpu');
        print('RAM: $_ram GB');
        print('Storage: $_storage GB');
        print('WiFi: ${_hasWifi ? 'Yes' : 'No'}');
        print('Bluetooth: ${_hasBluetooth ? 'Yes' : 'No'}');
        print('=====');
    }
}

class ComputerBuilder {
    final Computer _computer = Computer();
```

```
ComputerBuilder setCpu(String cpu) {
    _computer.setCpu(cpu);
    return this;
}

ComputerBuilder setGpu(String gpu) {
    _computer.setGpu(gpu);
    return this;
}

ComputerBuilder setRam(int ram) {
    _computer.setRam(ram);
    return this;
}

ComputerBuilder setStorage(int storage) {
    _computer.setStorage(storage);
    return this;
}

ComputerBuilder addWifi() {
    _computer.setWifi(true);
    return this;
}

ComputerBuilder addBluetooth() {
    _computer.setBluetooth(true);
    return this;
}

Computer build() {
    return _computer;
}

}

// Example: SQL Query Builder
class SQLQueryBuilder {
    String _query = '';

    SQLQueryBuilder select(String columns) {
        _query = 'SELECT $columns';
        return this;
    }

    SQLQueryBuilder from(String table) {
        _query += ' FROM $table';
        return this;
    }

    SQLQueryBuilder where(String condition) {
        _query += ' WHERE $condition';
        return this;
    }
}
```

```
SQLQueryBuilder orderBy(String column, {String direction = 'ASC'}) {
  _query += ' ORDER BY $column $direction';
  return this;
}

SQLQueryBuilder limit(int count) {
  _query += ' LIMIT $count';
  return this;
}

String build() {
  return '$_query';
}
}

// Demo code
void main() {
  print('=== Builder Pattern Demo ===\n');

  // Example 1: Classic Builder with Director
  print('--- Classic Builder with Director ---');

  var director = ConstructionDirector();

  // Build a simple house
  print('\nBuilding Simple House:');
  var simpleBuilder = SimpleHouseBuilder();
  director.setBuilder(simpleBuilder);
  director.buildBasicHouse();
  var simpleHouse = simpleBuilder.getResult();
  simpleHouse.display();

  // Build a luxury house
  print('\nBuilding Luxury House:');
  var luxuryBuilder = LuxuryHouseBuilder();
  director.setBuilder(luxuryBuilder);
  director.buildFullHouse();
  var luxuryHouse = luxuryBuilder.getResult();
  luxuryHouse.display();

  // Build a custom house
  print('\nBuilding Custom House (with garage and garden, no pool):');
  var customBuilder = SimpleHouseBuilder();
  director.setBuilder(customBuilder);
  director.buildCustomHouse(true, true, false);
  var customHouse = customBuilder.getResult();
  customHouse.display();

  // Example 2: Fluent Builder (method chaining)
  print('\n--- Fluent Builder (Method Chaining) ---');

  print('\nBuilding Gaming Computer:');
  var gamingComputer = ComputerBuilder()
    .setCpu('Intel Core i9-13900K')
```

```

        .setGpu('NVIDIA RTX 4090')
        .setRam(32)
        .setStorage(1000)
        .addWifi()
        .addBluetooth()
        .build();
gamingComputer.display();

print('\nBuilding Office Computer:');
var officeComputer = ComputerBuilder()
    .setCpu('Intel Core i5-13400')
    .setGpu('Integrated Graphics')
    .setRam(16)
    .setStorage(512)
    .addWifi()
    .build();
officeComputer.display();

// Example 3: SQL Query Builder
print('\n--- SQL Query Builder ---');

var query1 = SQLQueryBuilder()
    .select('*')
    .from('users')
    .where('age > 18')
    .orderBy('name', direction: 'ASC')
    .limit(10)
    .build();
print('Query 1: $query1');

var query2 = SQLQueryBuilder()
    .select('name, email')
    .from('customers')
    .where('active = true')
    .orderBy('created_at', direction: 'DESC')
    .build();
print('Query 2: $query2');
}

```

Prototype

Description: Creates new objects by **copying (cloning) an existing object**, rather than creating from scratch.

- **Why it is used:** When object creation is expensive or complex, cloning an existing object is faster and simpler. It avoids the overhead of re-initialization.
- **Where it is commonly used:** Game development (duplicating characters or levels), graphic editors (duplicating shapes), document templates.

Code Example (DART)

```

/*
 * PROTOTYPE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Prototype pattern specifies the kinds of objects to create using a
 * prototypical instance, and creates new objects by copying this prototype.
 * It allows you to create new objects by cloning existing ones instead of
 * creating them from scratch.
 *
 * WHY DOES IT EXIST?
 * - When creating an object is expensive or complex
 * - When you want to avoid creating subclasses just to create objects
 * - When you want to hide the concrete classes from the client
 * - When you need to create objects at runtime that are similar to
 * existing objects
 *
 * WHEN TO USE IT?
 * - When the classes to instantiate are specified at runtime
 * - When you want to avoid building a class hierarchy of factories that
 * parallels the class hierarchy of products
 * - When instances of a class can have one of only a few different
 * combinations of state
 * - When creating an object is expensive (e.g., requires database access,
 * network I/O, or complex initialization)
 * - Common use cases: game objects, document templates, graphic shapes,
 * configuration objects
 *
 * PROS:
 * - Allows you to clone objects without coupling to their concrete classes
 * - Can avoid repetitive initialization code
 * - Can be more efficient than creating objects from scratch
 * - Lets you add or remove products at runtime
 * - Works well with complex object graphs
 *
 * CONS:
 * - Cloning complex objects with circular references can be tricky
 * - Shallow vs. deep copying must be carefully considered
 * - May be difficult to implement if the object's internal state is not
 * easily accessible
 * - Can violate encapsulation if the clone method needs access to private
 * fields
 *
 * REAL-WORLD ANALOGY:
 * Think of a cell division in biology. A cell (prototype) divides to
 * create an identical copy of itself. The new cell starts with the same DNA
 * and characteristics as the original, but can then evolve independently.
 * Similarly, in software, you can clone an object and then modify it as
 * needed.
 */

// Prototype interface - declares the clone method
abstract class Prototype {

```

```
    Prototype clone();
    void display();
    String get type;
}

// Concrete Prototype 1: Rectangle
class Rectangle extends Prototype {
    int _width;
    int _height;
    String _color;

    Rectangle(this._width, this._height, this._color);

    // Clone method using named constructor for copying
    @override
    Rectangle clone() {
        return Rectangle(_width, _height, _color);
    }

    @override
    void display() {
        print('Rectangle: ${_width}x${_height}, Color: $_color');
    }

    @override
    String get type => 'Rectangle';

    // Methods to modify the cloned object
    void setWidth(int width) {
        _width = width;
    }

    void setHeight(int height) {
        _height = height;
    }

    void setColor(String color) {
        _color = color;
    }
}

// Concrete Prototype 2: Circle
class Circle extends Prototype {
    int _radius;
    String _color;

    Circle(this._radius, this._color);

    @override
    Circle clone() {
        return Circle(_radius, _color);
    }

    @override
```

```

void display() {
    print('Circle: Radius $_radius, Color: $_color');
}

@override
String get type => 'Circle';

void setRadius(int radius) {
    _radius = radius;
}

void setColor(String color) {
    _color = color;
}
}

// Prototype Registry - manages a collection of prototypes
class PrototypeRegistry {
    final Map<String, Prototype> _prototypes = {};

    void registerPrototype(String key, Prototype prototype) {
        _prototypes[key] = prototype;
    }

    Prototype? clone(String key) {
        var prototype = _prototypes[key];
        return prototype?.clone();
    }
}

// Example: Complex object with deep copy
class Employee extends Prototype {
    String _name;
    String _department;
    int _salary;
    List<String> _skills;

    Employee(this._name, this._department, this._salary, [List<String>?
skills])
        : _skills = skills ?? [];

    // Deep copy constructor
    @override
    Employee clone() {
        // Create a deep copy of the skills list
        var skillsCopy = List<String>.from(_skills);
        return Employee(_name, _department, _salary, skillsCopy);
    }

    @override
    void display() {
        print('Employee: $_name, Dept: $_department, Salary: \$_salary');
        print('Skills: ${_skills.join(', ')}');
    }
}

```

```
@override
String get type => 'Employee';

void addSkill(String skill) {
  _skills.add(skill);
}

void setName(String name) {
  _name = name;
}

void setSalary(int salary) {
  _salary = salary;
}
}

// Example: Document with shallow vs deep copy
class Document extends Prototype {
  String _title;
  String _content;

  Document(this._title, this._content);

  @override
  Document clone() {
    // For strings, this is effectively a deep copy since strings are
    // immutable in Dart
    return Document(_title, _content);
  }

  @override
  void display() {
    print('Document: $_title');
    print('Content: $_content');
  }

  @override
  String get type => 'Document';

  void appendContent(String text) {
    _content += text;
  }

  void setTitle(String title) {
    _title = title;
  }
}

// Example: Game Character with prototype
class GameCharacter extends Prototype {
  String _name;
  int _health;
  int _attack;
```

```

int _defense;
List<String> _inventory;

GameCharacter(this._name, this._health, this._attack, this._defense,
    [List<String>? inventory])
    : _inventory = inventory ?? [];

@override
GameCharacter clone() {
    return GameCharacter(
        _name, _health, _attack, _defense, List<String>.from(_inventory));
}

@override
void display() {
    print('Character: $_name');
    print('  Health: $_health, Attack: $_attack, Defense: $_defense');
    print('  Inventory: ${_inventory.join(', ')}');
}

@override
String get type => 'GameCharacter';

void addItem(String item) {
    _inventory.add(item);
}

void setName(String name) {
    _name = name;
}

void takeDamage(int amount) {
    _health = (_health - amount).clamp(0, double.infinity).toInt();
}
}

// Demo code
void main() {
    print('=== Prototype Pattern Demo ===\n');

    // Example 1: Basic Prototype with Registry
    print('--- Basic Prototype with Registry ---');

    var registry = PrototypeRegistry();

    // Register prototypes
    registry.registerPrototype('red_rectangle', Rectangle(10, 5, 'red'));
    registry.registerPrototype('blue_circle', Circle(7, 'blue'));

    // Clone and modify prototypes
    print('\nCloning red rectangle:');
    var rect1 = registry.clone('red_rectangle');
    rect1?.display();
}

```

```
print('\nCloning and modifying red rectangle:');
var rect2 = registry.clone('red_rectangle');
if (rect2 is Rectangle) {
  rect2.setWidth(20);
  rect2.setColor('green');
  rect2.display();
}

print('\nCloning blue circle:');
var circle1 = registry.clone('blue_circle');
circle1?.display();

// Example 2: Complex object with deep copy
print('\n--- Complex Object with Deep Copy ---');

var originalEmployee = Employee('John Doe', 'Engineering', 75000);
originalEmployee.addSkill('Dart');
originalEmployee.addSkill('Python');
originalEmployee.addSkill('JavaScript');

print('\nOriginal employee:');
originalEmployee.display();

print('\nCloned employee:');
var clonedEmployee = originalEmployee.clone();
clonedEmployee.setName('Jane Smith');
clonedEmployee.setSalary(80000);
clonedEmployee.addSkill('Go');
clonedEmployee.display();

print('\nOriginal employee (unchanged):');
originalEmployee.display();

// Example 3: Document with deep copy
print('\n--- Document with Deep Copy ---');

var originalDoc = Document('Project Plan', 'This document outlines the
project plan.');
```

```
print('\nOriginal document:');
originalDoc.display();

print('\nCloned document:');
var clonedDoc = originalDoc.clone();
clonedDoc.setTitle('Project Plan - Copy');
clonedDoc.appendContent(' It includes milestones and deliverables.');
```

```
clonedDoc.display();

print('\nOriginal document (unchanged):');
originalDoc.display();

// Example 4: Game Character with prototype
print('\n--- Game Character with Prototype ---');
```

```
var warriorTemplate = GameCharacter('Warrior', 100, 25, 15);
```

```
warriorTemplate.addItem('Sword');
warriorTemplate.addItem('Shield');

print('\nCreating warrior from template:');
var warrior1 = warriorTemplate.clone();
warrior1.setName('Arthur');
warrior1.display();

print('\nCreating another warrior from template:');
var warrior2 = warriorTemplate.clone();
warrior2.setName('Lancelot');
warrior2.takeDamage(20);
warrior2.addItem('Potion');
warrior2.display();

print('\nTemplate (unchanged):');
warriorTemplate.display();
}
```

2. Structural Design Patterns

Overview

Structural patterns deal with **how classes and objects are composed** to form larger structures. They simplify the structure by identifying relationships between entities and help ensure that when one part of a system changes, the entire structure does not need to change.

Adapter

Description: Acts as a **bridge between two incompatible interfaces**. It wraps an existing class with a new interface so that it becomes compatible with the client's expectations.

- **Why it is used:** To allow classes with incompatible interfaces to work together without modifying their source code.
 - **Where it is commonly used:** Integrating third-party libraries, legacy code integration, data format conversion (e.g., XML to JSON).
-

Code Example (DART)

```
/*
 * ADAPTER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Adapter pattern allows objects with incompatible interfaces to
 * collaborate. It acts as a bridge between two incompatible interfaces.
 *
 * WHY DOES IT EXIST?
 * - When you want to use an existing class whose interface doesn't match
```

the one you need

- * - When you want to create a reusable class that cooperates with unrelated classes with incompatible interfaces
- * - When you need to use several existing subclasses, but adapting each to match your interface would be impractical

*

* WHEN TO USE IT?

- * - When you want to use a third-party class but its interface doesn't match your system's interface
- * - When you need to integrate new components into an existing system without modifying existing code

* - When you want to reuse existing classes that have similar functionality but different interfaces

* - Common use cases: integrating third-party libraries, legacy code integration, data format conversion

*

* PROS:

- * - Single Responsibility Principle (separates conversion logic from business logic)
- * - Open/Closed Principle (introduce new adapters without modifying existing code)
- * - Allows incompatible classes to work together
- * - Improves code reusability

*

* CONS:

- * - Increases overall code complexity
- * - Can introduce additional layers of indirection
- * - May not be suitable for simple cases where direct modification is easier
- * - Can make the code harder to understand due to multiple layers

*

* REAL-WORLD ANALOGY:

* Think of a power adapter when traveling abroad. Your device (client) expects a certain plug shape (target interface), but the wall outlet (adaptee) has a different shape. The adapter converts between the two, allowing your device to work with the foreign outlet without modifying either the device or the wall.

*/

// Target Interface - the interface that the client expects

```
abstract class MediaPlayer {
  void play(String audioType, String fileName);
}
```

// Adaptee - the class with an incompatible interface

```
class AdvancedMediaPlayer {
  void playVlc(String fileName) {
    print('Playing vlc file: $fileName');
  }

  void playMp4(String fileName) {
    print('Playing mp4 file: $fileName');
  }
}
```

```
// Adapter - makes the Adaptee's interface compatible with the Target
interface
class MediaAdapter implements MediaPlayer {
    final AdvancedMediaPlayer _advancedMusicPlayer = AdvancedMediaPlayer();

    @override
    void play(String audioType, String fileName) {
        if (audioType == 'vlc') {
            _advancedMusicPlayer.playVlc(fileName);
        } else if (audioType == 'mp4') {
            _advancedMusicPlayer.playMp4(fileName);
        } else {
            print('Invalid media. $audioType format not supported');
        }
    }
}

// Client - uses the Target interface
class AudioPlayer implements MediaPlayer {
    MediaAdapter? _mediaAdapter;

    @override
    void play(String audioType, String fileName) {
        // Built-in support for mp3
        if (audioType == 'mp3') {
            print('Playing mp3 file: $fileName');
        }
        // MediaAdapter provides support for other formats
        else if (audioType == 'vlc' || audioType == 'mp4') {
            _mediaAdapter = MediaAdapter();
            _mediaAdapter!.play(audioType, fileName);
        } else {
            print('Invalid media. $audioType format not supported');
        }
    }
}

// Example 2: Payment Processing Adapter
// This example shows a different scenario: payment processing

// Target Interface
abstract class PaymentProcessor {
    void processPayment(double amount);
}

// Adaptee - incompatible payment system
class StripePaymentSystem {
    void makeStripePayment(double amount) {
        print('Processing $$amount via Stripe');
    }
}

// Adaptee - another incompatible payment system
```

```
class PayPalPaymentSystem {
  void sendPayPalRequest(double amount) {
    print('Processing \$$amount via PayPal');
  }
}

// Adapter for Stripe
class StripeAdapter implements PaymentProcessor {
  final StripePaymentSystem _stripeSystem = StripePaymentSystem();

  @override
  void processPayment(double amount) {
    _stripeSystem.makeStripePayment(amount);
  }
}

// Adapter for PayPal
class PayPalAdapter implements PaymentProcessor {
  final PayPalPaymentSystem _paypalSystem = PayPalPaymentSystem();

  @override
  void processPayment(double amount) {
    _paypalSystem.sendPayPalRequest(amount);
  }
}

// Example 3: Two-way Adapter
// Allows both interfaces to work with each other

// Interface A
abstract class CelsiusTemperature {
  double getTemperatureInCelsius();
  void setTemperatureInCelsius(double temp);
}

// Interface B
abstract class FahrenheitTemperature {
  double getTemperatureInFahrenheit();
  void setTemperatureInFahrenheit(double temp);
}

// Two-way adapter
class TemperatureAdapter implements CelsiusTemperature,
FahrenheitTemperature {
  double _temperatureInCelsius;

  TemperatureAdapter([this._temperatureInCelsius = 0.0]);

  // Celsius interface implementation
  @override
  double getTemperatureInCelsius() {
    return _temperatureInCelsius;
  }
}
```

```

@override
void setTemperatureInCelsius(double temp) {
  _temperatureInCelsius = temp;
}

// Fahrenheit interface implementation
@override
double getTemperatureInFahrenheit() {
  return (_temperatureInCelsius * 9 / 5) + 32;
}

@override
void setTemperatureInFahrenheit(double temp) {
  _temperatureInCelsius = (temp - 32) * 5 / 9;
}
}

// Demo code
void main() {
  print('=== Adapter Pattern Demo ===\n');

  // Example 1: Media Player Adapter
  print('--- Media Player Adapter ---');

  var audioPlayer = AudioPlayer();

  audioPlayer.play('mp3', 'beyond the horizon.mp3');
  audioPlayer.play('mp4', 'alone again.mp4');
  audioPlayer.play('vlc', 'far far away.vlc');
  audioPlayer.play('avi', 'mind me.avi');

  print('');

  // Example 2: Payment Processing Adapter
  print('--- Payment Processing Adapter ---');

  PaymentProcessor stripeAdapter = StripeAdapter();
  stripeAdapter.processPayment(100.50);

  PaymentProcessor paypalAdapter = PayPalAdapter();
  paypalAdapter.processPayment(75.25);

  print('');

  // Example 3: Two-way Temperature Adapter
  print('--- Two-way Temperature Adapter ---');

  var tempAdapter = TemperatureAdapter();

  // Set using Celsius interface
  tempAdapter.setTemperatureInCelsius(25.0);
  print('Temperature in Celsius: ${tempAdapter.getTemperatureInCelsius()} °C');
  print('Temperature in Fahrenheit:

```

```

    ${tempAdapter.getTemperatureInFahrenheit()}°F');

    // Set using Fahrenheit interface
    tempAdapter.setTemperatureInFahrenheit(77.0);
    print('\nTemperature in Celsius: ${tempAdapter.getTemperatureInCelsius()}
°C');
    print('Temperature in Fahrenheit:
    ${tempAdapter.getTemperatureInFahrenheit()}°F');
}

```

Bridge

Description: Decouples an abstraction from its implementation so that the two can vary independently.

- **Why it is used:** To avoid a permanent binding between an abstraction and its implementation and to allow both to be extended independently.
- **Where it is commonly used:** Device driver systems, rendering engines (OpenGL vs. DirectX), platform-independent UI frameworks.

Code Example (DART)

```

/*
 * BRIDGE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Bridge pattern separates abstraction from implementation so that
both can vary independently.
 * It uses composition instead of inheritance to decouple the abstraction
from its implementation.
 *
 * WHY DOES IT EXIST?
 * - When you want to avoid a permanent binding between an abstraction and
its implementation
 * - When both the abstraction and the implementation should be extensible
by subclassing
 * - When changes in the implementation should not affect clients
 * - When you have a class hierarchy that explodes due to combining
multiple dimensions
 *
 * WHEN TO USE IT?
 * - When you want to split a monolithic class that has several variants of
some functionality
 * - When you need to extend a class in several independent dimensions
 * - When you want to be able to switch implementations at runtime
 * - Common use cases: UI frameworks (different controls on different
platforms), shape rendering, device drivers
 *
 * PROS:
 * - Separates abstraction from implementation
 * - Improves extensibility (can extend abstractions and implementations

```

```

independently)
* - Hides implementation details from clients
* - Reduces the number of subclasses needed (avoids combinatorial
explosion)
*
* CONS:
* - Increases code complexity
* - May require additional indirection
* - Can make the code harder to understand initially
* - Might be overkill for simple scenarios
*
* REAL-WORLD ANALOGY:
* Think of a TV remote control (abstraction) and the TV itself
(implementation). The remote control doesn't need to know the internal
workings of the TV. You can use the same remote with different TV brands,
and different remotes with the same TV. The remote (abstraction) is bridged
to the TV (implementation) through a standard interface (infrared signals).
*/

// Implementation Interface - defines the operations for all concrete
implementations
abstract class DrawingAPI {
  void drawCircle(double x, double y, double radius);
  void drawRectangle(double x, double y, double width, double height);
}

// Concrete Implementation 1: Red Circle API
class RedCircleAPI implements DrawingAPI {
  @override
  void drawCircle(double x, double y, double radius) {
    print('Drawing Circle [color: red, center: ($x, $y), radius:
$radius]');
  }

  @override
  void drawRectangle(double x, double y, double width, double height) {
    print('Drawing Rectangle [color: red, top-left: ($x, $y), width:
$width, height: $height]');
  }
}

// Concrete Implementation 2: Green Circle API
class GreenCircleAPI implements DrawingAPI {
  @override
  void drawCircle(double x, double y, double radius) {
    print('Drawing Circle [color: green, center: ($x, $y), radius:
$radius]');
  }

  @override
  void drawRectangle(double x, double y, double width, double height) {
    print('Drawing Rectangle [color: green, top-left: ($x, $y), width:
$width, height: $height]');
  }
}

```

```

}

// Abstraction - defines the abstraction's interface and maintains a
reference to an implementation
abstract class Shape {
  final DrawingAPI drawingAPI;

  Shape(this.drawingAPI);

  void draw();
  void resize(double percentage);
}

// Refined Abstraction 1: Circle
class Circle extends Shape {
  double _x, _y, _radius;

  Circle(this._x, this._y, this._radius, DrawingAPI api) : super(api);

  @override
  void draw() {
    drawingAPI.drawCircle(_x, _y, _radius);
  }

  @override
  void resize(double percentage) {
    _radius *= percentage / 100;
    print('Resizing circle to $_radius radius');
  }
}

// Refined Abstraction 2: Rectangle
class Rectangle extends Shape {
  double _x, _y, _width, _height;

  Rectangle(this._x, this._y, this._width, this._height, DrawingAPI api) :
super(api);

  @override
  void draw() {
    drawingAPI.drawRectangle(_x, _y, _width, _height);
  }

  @override
  void resize(double percentage) {
    _width *= percentage / 100;
    _height *= percentage / 100;
    print('Resizing rectangle to ${_width}x$_height');
  }
}

// Example 2: Device and Remote Control Bridge
// This shows how the Bridge pattern can be used for device control

```

```
// Implementation Interface
abstract class Device {
  void turnOn();
  void turnOff();
  void setVolume(int volume);
  int getVolume();
}

// Concrete Implementation 1: TV
class Television implements Device {
  bool _on = false;
  int _volume = 10;

  @override
  void turnOn() {
    _on = true;
    print('TV is now ON');
  }

  @override
  void turnOff() {
    _on = false;
    print('TV is now OFF');
  }

  @override
  void setVolume(int volume) {
    if (_on) {
      _volume = volume;
      print('TV volume set to $volume');
    } else {
      print('Cannot set volume: TV is OFF');
    }
  }

  @override
  int getVolume() {
    return _volume;
  }
}

// Concrete Implementation 2: Radio
class Radio implements Device {
  bool _on = false;
  int _volume = 5;

  @override
  void turnOn() {
    _on = true;
    print('Radio is now ON');
  }

  @override
  void turnOff() {
```

```

    _on = false;
    print('Radio is now OFF');
}

@override
void setVolume(int volume) {
    if (_on) {
        _volume = volume;
        print('Radio volume set to $volume');
    } else {
        print('Cannot set volume: Radio is OFF');
    }
}

@override
int getVolume() {
    return _volume;
}
}

// Abstraction: Remote Control
class RemoteControl {
    final Device _device;

    RemoteControl(this._device);

    void togglePower() {
        if (_device.getVolume() >= 0) {
            _device.turnOff();
        } else {
            _device.turnOn();
        }
    }

    void volumeUp() {
        int currentVolume = _device.getVolume();
        _device.setVolume(currentVolume + 1);
    }

    void volumeDown() {
        int currentVolume = _device.getVolume();
        _device.setVolume(currentVolume - 1);
    }
}

// Refined Abstraction: Advanced Remote Control
class AdvancedRemoteControl extends RemoteControl {
    AdvancedRemoteControl(Device device) : super(device);

    void mute() {
        print('Muting device');
        _device.setVolume(0);
    }
}

```

```
// Demo code
void main() {
    print('=== Bridge Pattern Demo ===\n');

    // Example 1: Shape Drawing Bridge
    print('--- Shape Drawing Bridge ---');

    var redCircle = Circle(100, 100, 10, RedCircleAPI());
    var greenCircle = Circle(100, 100, 10, GreenCircleAPI());

    redCircle.draw();
    greenCircle.draw();

    var redRectangle = Rectangle(50, 50, 20, 30, RedCircleAPI());
    var greenRectangle = Rectangle(50, 50, 20, 30, GreenCircleAPI());

    redRectangle.draw();
    greenRectangle.draw();

    print('\nResizing shapes:');
    redCircle.resize(150);
    redCircle.draw();

    print('');

    // Example 2: Device and Remote Control Bridge
    print('--- Device and Remote Control Bridge ---');

    var tv = Television();
    var tvRemote = RemoteControl(tv);

    print('\nControlling TV with basic remote:');
    tvRemote.togglePower();
    tvRemote.volumeUp();
    tvRemote.volumeUp();
    tvRemote.volumeDown();
    tvRemote.togglePower();

    var radio = Radio();
    var advancedRemote = AdvancedRemoteControl(radio);

    print('\nControlling Radio with advanced remote:');
    advancedRemote.togglePower();
    advancedRemote.volumeUp();
    advancedRemote.mute();
    advancedRemote.togglePower();
}
```

Composite

Description: Composes objects into **tree structures to represent part-whole hierarchies**. It lets clients treat individual objects and compositions of objects uniformly.

- **Why it is used:** To simplify client code that deals with tree structures by treating individual items and groups in the same way.
- **Where it is commonly used:** File system structures (files and folders), UI component trees, organization hierarchies.

Code Example (DART)

```
/*
 * COMPOSITE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Composite pattern lets you compose objects into tree structures and
work with these structures as if they were individual objects.
 * It treats individual objects and compositions of objects uniformly.
 *
 * WHY DOES IT EXIST?
 * - When you want to represent part-whole hierarchies of objects
 * - When you want clients to ignore the difference between compositions of
objects and individual objects
 * - When you want to treat individual objects and compositions uniformly
 *
 * WHEN TO USE IT?
 * - When you have a tree structure of objects (e.g., file systems,
organization charts, UI component hierarchies)
 * - When you want to simplify client code by treating individual and
composite objects the same way
 * - When the structure of your objects forms a recursive hierarchy
 * - Common use cases: file systems, graphics scenes, UI component trees,
organization structures
 *
 * PROS:
 * - Simplifies client code (can treat complex and simple objects
uniformly)
 * - Open/Closed Principle (easy to add new kinds of components)
 * - Flexible tree structure (can build complex structures from simple
parts)
 * - Makes it easier to add new types of components
 *
 * CONS:
 * - Can make the design overly general (hard to restrict components)
 * - May be difficult to restrict the types of components in a composite
 * - Can make the code harder to understand due to recursion
 * - Performance overhead for traversing large trees
 *
 * REAL-WORLD ANALOGY:
 * Think of a file system. A file is a simple object (leaf), while a folder
is a composite that can contain both files and other folders. When you
delete a folder, you want to delete everything inside it recursively. The
```

```

file system treats files and folders uniformly in many operations (like
copying, moving, or calculating size).
*/

// Component Interface - declares the interface for objects in the
composition
abstract class FileSystemComponent {
  void display([String indent = '']);
  int getSize();

  void add(FileSystemComponent component) {
    print('Cannot add to a file');
  }

  void remove(FileSystemComponent component) {
    print('Cannot remove from a file');
  }

  FileSystemComponent? getChild(int index) {
    return null;
  }
}

// Leaf - represents individual objects in the composition
class FileComponent extends FileSystemComponent {
  final String name;
  final int size;

  FileComponent(this.name, this.size);

  @override
  void display([String indent = '']) {
    print('$indent File: $name ($size bytes)');
  }

  @override
  int getSize() {
    return size;
  }
}

// Composite - represents groups of objects that can contain other
composites
class Directory extends FileSystemComponent {
  final String name;
  final List<FileSystemComponent> _children = [];

  Directory(this.name);

  @override
  void add(FileSystemComponent component) {
    _children.add(component);
  }
}

```

```

@override
void remove(FileSystemComponent component) {
  _children.remove(component);
}

@override
FileSystemComponent? getChild(int index) {
  if (index >= 0 && index < _children.length) {
    return _children[index];
  }
  return null;
}

@override
void display([String indent = '']) {
  print('$indent Directory: $name (${getSize()} bytes)');
  for (var child in _children) {
    child.display(indent + ' ');
  }
}

@override
int getSize() {
  int totalSize = 0;
  for (var child in _children) {
    totalSize += child.getSize();
  }
  return totalSize;
}
}

// Example 2: Organization Structure
// Shows how Composite can be used for organizational hierarchies

abstract class Employee {
  void print([String indent = '']);

  void add(Employee employee) {
    print('Cannot add to a regular employee');
  }

  void remove(Employee employee) {
    print('Cannot remove from a regular employee');
  }
}

class Worker extends Employee {
  final String name;
  final String position;

  Worker(this.name, this.position);

  @override
  void print([String indent = '']) {

```

```
        print('$indent Employee: $name - $position');
    }
}

class Manager extends Employee {
    final String name;
    final String title;
    final List<Employee> _subordinates = [];

    Manager(this.name, this.title);

    @override
    void add(Employee employee) {
        _subordinates.add(employee);
    }

    @override
    void remove(Employee employee) {
        _subordinates.remove(employee);
    }

    @override
    void print([String indent = '']) {
        print('$indent Manager: $name - $title');
        for (var subordinate in _subordinates) {
            subordinate.print(indent + ' ');
        }
    }
}

// Example 3: Graphic Shapes Composite
// Shows how Composite can be used for graphics

abstract class Graphic {
    void draw([String indent = '']);
}

class CircleGraphic extends Graphic {
    final String name;

    CircleGraphic(this.name);

    @override
    void draw([String indent = '']) {
        print('$indent Drawing Circle: $name');
    }
}

class SquareGraphic extends Graphic {
    final String name;

    SquareGraphic(this.name);

    @override
```

```
void draw([String indent = '']) {
  print('$indent Drawing Square: $name');
}

class GraphicGroup extends Graphic {
  final String name;
  final List<Graphic> _graphics = [];

  GraphicGroup(this.name);

  void add(Graphic graphic) {
    _graphics.add(graphic);
  }

  void remove(Graphic graphic) {
    _graphics.remove(graphic);
  }

  @override
  void draw([String indent = '']) {
    print('$indent Group: $name');
    for (var graphic in _graphics) {
      graphic.draw(indent + ' ');
    }
  }
}

// Demo code
void main() {
  print('=== Composite Pattern Demo ===\n');

  // Example 1: File System
  print('--- File System Example ---');

  var file1 = FileComponent('file1.txt', 100);
  var file2 = FileComponent('file2.txt', 200);
  var file3 = FileComponent('file3.txt', 150);

  var dir1 = Directory('Documents');
  var dir2 = Directory('Pictures');
  var rootDir = Directory('Root');

  dir1.add(file1);
  dir1.add(file2);

  dir2.add(file3);

  rootDir.add(dir1);
  rootDir.add(dir2);

  print('\nFile System Structure:');
  rootDir.display();
}
```

```
print('\nTotal size of Root directory: ${rootDir.getSize()} bytes');

print('');

// Example 2: Organization Structure
print('--- Organization Structure Example ---');

var ceo = Manager('John Smith', 'CEO');
var cto = Manager('Alice Johnson', 'CTO');
var cfo = Manager('Bob Williams', 'CFO');

var dev1 = Worker('Developer 1', 'Senior Developer');
var dev2 = Worker('Developer 2', 'Junior Developer');
var accountant = Worker('Accountant', 'Senior Accountant');

cto.add(dev1);
cto.add(dev2);
cfo.add(accountant);

ceo.add(cto);
ceo.add(cfo);

print('\nOrganization Structure:');
ceo.print();

print('');

// Example 3: Graphic Shapes
print('--- Graphic Shapes Example ---');

var circle1 = CircleGraphic('Red Circle');
var circle2 = CircleGraphic('Blue Circle');
var square1 = SquareGraphic('Green Square');

var group1 = GraphicGroup('Shapes Group 1');
var group2 = GraphicGroup('Shapes Group 2');
var mainGroup = GraphicGroup('Main Group');

group1.add(circle1);
group1.add(square1);

group2.add(circle2);

mainGroup.add(group1);
mainGroup.add(group2);

print('\nGraphic Structure:');
mainGroup.draw();
}
```

Decorator

Description: Adds new behavior or responsibilities to an object dynamically, without altering its class. It wraps the original object and adds functionality on top.

- **Why it is used:** To extend object functionality at runtime without using inheritance, keeping classes flexible and following the Open/Closed Principle.
- **Where it is commonly used:** I/O streams in Java, middleware in web frameworks, text formatting systems.

Code Example (DART)

```
/*
 * DECORATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Decorator pattern lets you attach new behaviors to objects by
 * placing these objects inside special wrapper objects that contain the
 * behaviors.
 * It provides a flexible alternative to inheritance for extending
 * functionality.
 *
 * WHY DOES IT EXIST?
 * - When you want to add responsibilities to individual objects
 *   dynamically without affecting other objects
 * - When extending a class by subclassing is impractical (too many
 *   subclasses)
 * - When you want to add and remove responsibilities at runtime
 *
 * WHEN TO USE IT?
 * - When you need to add responsibilities to individual objects without
 *   affecting other objects of the same class
 * - When the class definition may be hidden or otherwise unavailable for
 *   subclassing
 * - When you want to add functionality to objects in a transparent way
 *   (clients shouldn't know the difference)
 * - Common use cases: UI components (scrollbars, borders), data streams
 *   (compression, encryption), logging, caching
 *
 * PROS:
 * - Single Responsibility Principle (divides functionality into small
 *   classes)
 * - Open/Closed Principle (can extend behavior without modifying existing
 *   code)
 * - Flexible alternative to subclassing
 * - Can add/remove responsibilities at runtime
 *
 * CONS:
 * - Can result in many small classes (code complexity)
 * - Can make it hard to remove wrapped decorators
 * - Can make debugging difficult due to many layers
 * - Initial setup can be complex
 */
```

```
* REAL-WORLD ANALOGY:
* Think of ordering coffee. You start with a basic coffee (component),
then you can add decorators like milk, sugar, whipped cream, or syrup. Each
decorator adds its own behavior (flavor, cost) while still being treated as
coffee. You can combine multiple decorators in any order.
*/

// Component Interface - defines the interface for objects that can have
responsibilities added
abstract class Coffee {
    double getCost();
    String getDescription();
}

// Concrete Component - the basic object that can have responsibilities
added
class SimpleCoffee extends Coffee {
    @override
    double getCost() {
        return 2.0;
    }

    @override
    String getDescription() {
        return 'Simple Coffee';
    }
}

// Decorator - implements the Component interface and wraps a Component
abstract class CoffeeDecorator extends Coffee {
    final Coffee decoratedCoffee;

    CoffeeDecorator(this.decoratedCoffee);

    @override
    double getCost() {
        return decoratedCoffee.getCost();
    }

    @override
    String getDescription() {
        return decoratedCoffee.getDescription();
    }
}

// Concrete Decorator 1: Milk
class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(super.coffee);

    @override
    double getCost() {
        return super.getCost() + 0.5;
    }
}
```

```
@override
String getDescription() {
    return '${super.getDescription()}, Milk';
}

// Concrete Decorator 2: Sugar
class SugarDecorator extends CoffeeDecorator {
    SugarDecorator(super.coffee);

    @override
    double getCost() {
        return super.getCost() + 0.2;
    }

    @override
    String getDescription() {
        return '${super.getDescription()}, Sugar';
    }
}

// Concrete Decorator 3: Whipped Cream
class WhippedCreamDecorator extends CoffeeDecorator {
    WhippedCreamDecorator(super.coffee);

    @override
    double getCost() {
        return super.getCost() + 1.0;
    }

    @override
    String getDescription() {
        return '${super.getDescription()}, Whipped Cream';
    }
}

// Example 2: Text Formatting Decorator
// Shows how decorators can be used for text processing

abstract class TextComponent {
    String getText();
}

class SimpleText extends TextComponent {
    final String text;

    SimpleText(this.text);

    @override
    String getText() {
        return text;
    }
}
```

```
abstract class TextDecorator extends TextComponent {
    final TextComponent wrappedText;

    TextDecorator(this.wrappedText);

    @override
    String getText() {
        return wrappedText.getText();
    }
}

class BoldDecorator extends TextDecorator {
    BoldDecorator(super.text);

    @override
    String getText() {
        return '<b>${super.getText()}</b>';
    }
}

class ItalicDecorator extends TextDecorator {
    ItalicDecorator(super.text);

    @override
    String getText() {
        return '<i>${super.getText()}</i>';
    }
}

class UnderlineDecorator extends TextDecorator {
    UnderlineDecorator(super.text);

    @override
    String getText() {
        return '<u>${super.getText()}</u>';
    }
}

// Example 3: Data Stream Decorator
// Shows how decorators can be used for data processing

abstract class DataSource {
    void writeData(String data);
    String readData();
}

class FileDataSource extends DataSource {
    String _data = '';

    @override
    void writeData(String data) {
        _data = data;
        print('Writing data to file: $data');
    }
}
```

```
@override
String readData() {
    print('Reading data from file: $_data');
    return _data;
}

}

abstract class DataSourceDecorator extends DataSource {
    final DataSource wrappedSource;

    DataSourceDecorator(this.wrappedSource);

    @override
    void writeData(String data) {
        wrappedSource.writeData(data);
    }

    @override
    String readData() {
        return wrappedSource.readData();
    }
}

class EncryptionDecorator extends DataSourceDecorator {
    EncryptionDecorator(super.source);

    @override
    void writeData(String data) {
        var encrypted = '[ENCRYPTED]$data[ENCRYPTED]';
        print('Encrypting data...');
        wrappedSource.writeData(encrypted);
    }

    @override
    String readData() {
        var data = wrappedSource.readData();
        print('Decrypting data...');
        // Remove encryption markers for demo
        if (data.startsWith('[ENCRYPTED]')) {
            return data.substring(11, data.length - 11);
        }
        return data;
    }
}

class CompressionDecorator extends DataSourceDecorator {
    CompressionDecorator(super.source);

    @override
    void writeData(String data) {
        var compressed = '[COMPRESSED]$data[COMPRESSED]';
        print('Compressing data...');
        wrappedSource.writeData(compressed);
    }
}
```

```
}

@override
String readData() {
  var data = wrappedSource.readData();
  print('Decompressing data...');
  // Remove compression markers for demo
  if (data.startsWith('[COMPRESSED]')) {
    return data.substring(12, data.length - 12);
  }
  return data;
}
}

// Demo code
void main() {
  print('=== Decorator Pattern Demo ===\n');

  // Example 1: Coffee Shop
  print('--- Coffee Shop Example ---');

  Coffee coffee = SimpleCoffee();
  print('${coffee.getDescription()} \${coffee.getCost()}');

  coffee = MilkDecorator(coffee);
  print('${coffee.getDescription()} \${coffee.getCost()}');

  coffee = SugarDecorator(coffee);
  print('${coffee.getDescription()} \${coffee.getCost()}');

  coffee = WhippedCreamDecorator(coffee);
  print('${coffee.getDescription()} \${coffee.getCost()}');

  print('');

  // Example 2: Text Formatting
  print('--- Text Formatting Example ---');

  TextComponent text = SimpleText('Hello World');
  print('Original: ${text.getText()}');

  text = BoldDecorator(text);
  print('Bold: ${text.getText()}');

  text = ItalicDecorator(text);
  print('Bold + Italic: ${text.getText()}');

  text = UnderlineDecorator(text);
  print('Bold + Italic + Underline: ${text.getText()}');

  print('');

  // Example 3: Data Stream Processing
  print('--- Data Stream Processing Example ---');
```

```

DataSource dataSource = FileDataSource();
print('\nWriting plain data:');
dataSource.writeData('Important Data');
print('Reading: ${dataSource.readData()}');

print('\nWriting encrypted data:');
dataSource = EncryptionDecorator(dataSource);
dataSource.writeData('Secret Data');
print('Reading: ${dataSource.readData()}');

print('\nWriting encrypted and compressed data:');
dataSource = CompressionDecorator(dataSource);
dataSource.writeData('Very Secret Data');
print('Reading: ${dataSource.readData()}');
}

```

Facade

Description: Provides a **simplified, unified interface** to a complex subsystem or set of interfaces.

- **Why it is used:** To hide complexity from the client. The client interacts with a simple interface instead of dealing with multiple complex components.
- **Where it is commonly used:** API wrappers, library interfaces, startup/shutdown systems in complex applications.

Code Example (DART)

```

/*
 * FACADE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Facade pattern provides a simplified interface to a library, a
 * framework, or any other complex set of classes.
 * It hides the complexities of the larger system and provides a simpler
 * interface to the client.
 *
 * WHY DOES IT EXIST?
 * - When you need to provide a simple interface to a complex subsystem
 * - When you want to reduce dependencies between clients and the
 * implementation of an abstraction
 * - When you want to layer your subsystems (use facade to define entry
 * points)
 *
 * WHEN TO USE IT?
 * - When you have a complex system with many interacting classes
 * - When you want to provide a simple interface for clients to interact
 * with the system
 * - When you want to decouple the client from the subsystem implementation
 * - Common use cases: API libraries, database operations, complex UI
 */

```

```

systems, payment processing
*
* PROS:
* - Simplifies the interface to a complex subsystem
* - Decouples clients from the subsystem (reduces coupling)
* - Makes the system easier to use and understand
* - Promotes loose coupling between subsystems and clients
*
* CONS:
* - Can become a "god object" if not carefully designed
* - May limit the flexibility of the subsystem
* - Can hide useful functionality from advanced users
* - Adds an extra layer of indirection
*
* REAL-WORLD ANALOGY:
* Think of a car dashboard. The dashboard (facade) provides a simple
interface to the complex car systems (engine, transmission, brakes, etc.).
You don't need to know how the engine works to drive the car - you just use
the steering wheel, pedals, and dashboard controls. The dashboard hides the
complexity while giving you access to the essential functions.
*/

// Subsystem Class 1: CPU
class CPU {
  void freeze() {
    print('CPU: Freezing...');
  }

  void jump(int position) {
    print('CPU: Jumping to position $position');
  }

  void execute() {
    print('CPU: Executing instructions...');
  }
}

// Subsystem Class 2: Memory
class Memory {
  void load(int position, String data) {
    print('Memory: Loading data \'$data\' at position $position');
  }
}

// Subsystem Class 3: Hard Drive
class HardDrive {
  String read(int lba, int size) {
    print('HardDrive: Reading $size bytes from LBA $lba');
    return 'Boot Data';
  }
}

// Facade - provides a simplified interface to the complex subsystem
class ComputerFacade {

```

```
final CPU _cpu = CPU();
final Memory _memory = Memory();
final HardDrive _hardDrive = HardDrive();

void start() {
    print('=== Starting Computer ===');
    _cpu.freeze();
    _memory.load(0, _hardDrive.read(0, 1024));
    _cpu.jump(0);
    _cpu.execute();
    print('=== Computer Started ===');
}

void shutdown() {
    print('=== Shutting Down Computer ===');
    print('CPU: Stopping...');
    print('Memory: Clearing...');
    print('HardDrive: Spinning down...');
    print('=== Computer Shutdown ===');
}

// Example 2: Home Theater System
// Shows how Facade can simplify controlling multiple devices

class Amplifier {
    void on() => print('Amplifier: ON');
    void off() => print('Amplifier: OFF');
    void setVolume(int level) => print('Amplifier: Volume set to $level');
}

class Tuner {
    void on() => print('Tuner: ON');
    void off() => print('Tuner: OFF');
    void setFrequency(double freq) => print('Tuner: Frequency set to $freq FM');
}

class DVDPlayer {
    void on() => print('DVD Player: ON');
    void off() => print('DVD Player: OFF');
    void play(String movie) => print('DVD Player: Playing \'$movie\'');
}

class Projector {
    void on() => print('Projector: ON');
    void off() => print('Projector: OFF');
    void setInput(String input) => print('Projector: Input set to $input');
}

class TheaterLights {
    void on() => print('Theater Lights: ON');
    void off() => print('Theater Lights: OFF');
    void dim(int level) => print('Theater Lights: Dimmed to $level%');
```

```
}

class ProjectionScreen {
  void down() => print('Screen: DOWN');
  void up() => print('Screen: UP');
}

class PopcornPopper {
  void on() => print('Popcorn Popper: ON');
  void off() => print('Popcorn Popper: OFF');
  void pop() => print('Popcorn Popper: Popping popcorn!');
}

// Facade for Home Theater
class HomeTheaterFacade {
  final Amplifier _amp = Amplifier();
  final Tuner _tuner = Tuner();
  final DVDPlayer _dvd = DVDPlayer();
  final Projector _projector = Projector();
  final TheaterLights _lights = TheaterLights();
  final ProjectionScreen _screen = ProjectionScreen();
  final PopcornPopper _popper = PopcornPopper();

  void watchMovie(String movie) {
    print('\n=== Get ready to watch a movie ===');
    _popper.on();
    _popper.pop();
    _lights.dim(10);
    _screen.down();
    _projector.on();
    _projector.setInput('DVD');
    _amp.on();
    _amp.setVolume(5);
    _dvd.on();
    _dvd.play(movie);
    print('=== Movie is playing ===');
  }

  void endMovie() {
    print('\n=== Shutting movie theater down ===');
    _popper.off();
    _lights.on();
    _screen.up();
    _projector.off();
    _amp.off();
    _dvd.off();
    print('=== Movie theater shut down ===');
  }

  void listenToRadio(double frequency) {
    print('\n=== Tuning in to radio ===');
    _amp.on();
    _amp.setVolume(5);
    _tuner.on();
  }
}
```

```
    _tuner.setFrequency(frequency);
    print('=== Radio is playing ===');
}

void turnOffRadio() {
    print('\n=== Shutting down radio ===');
    _tuner.off();
    _amp.off();
    print('=== Radio turned off ===');
}
}

// Example 3: Order Processing System
// Shows how Facade can simplify complex business logic

class InventorySystem {
    bool checkInventory(String item, int quantity) {
        print('Inventory: Checking $quantity units of $item');
        return true; // Simplified for demo
    }

    void updateInventory(String item, int quantity) {
        print('Inventory: Updating $quantity units of $item');
    }
}

class PaymentSystem {
    bool processPayment(double amount) {
        print('Payment: Processing payment of \$$amount');
        return true; // Simplified for demo
    }

    void refundPayment(double amount) {
        print('Payment: Refunding \$$amount');
    }
}

class ShippingSystem {
    void scheduleShipping(String address) {
        print('Shipping: Scheduling delivery to $address');
    }

    void cancelShipping() {
        print('Shipping: Cancelling delivery');
    }
}

class NotificationSystem {
    void sendConfirmation(String email) {
        print('Notification: Sending confirmation to $email');
    }

    void sendCancellation(String email) {
        print('Notification: Sending cancellation to $email');
    }
}
```

```
}  
}  
  
// Facade for Order Processing  
class OrderFacade {  
  final InventorySystem _inventory = InventorySystem();  
  final PaymentSystem _payment = PaymentSystem();  
  final ShippingSystem _shipping = ShippingSystem();  
  final NotificationSystem _notification = NotificationSystem();  
  
  bool placeOrder(String item, int quantity, double amount,  
                  String address, String email) {  
    print('\n=== Processing Order ===');  
  
    if (!_inventory.checkInventory(item, quantity)) {  
      print('Order failed: Item not in stock');  
      return false;  
    }  
  
    if (!_payment.processPayment(amount)) {  
      print('Order failed: Payment declined');  
      return false;  
    }  
  
    _inventory.updateInventory(item, -quantity);  
    _shipping.scheduleShipping(address);  
    _notification.sendConfirmation(email);  
  
    print('=== Order Placed Successfully ===');  
    return true;  
  }  
  
  void cancelOrder(String item, int quantity, double amount, String email)  
  {  
    print('\n=== Cancelling Order ===');  
    _inventory.updateInventory(item, quantity);  
    _payment.refundPayment(amount);  
    _shipping.cancelShipping();  
    _notification.sendCancellation(email);  
    print('=== Order Cancelled ===');  
  }  
}  
  
// Demo code  
void main() {  
  print('=== Facade Pattern Demo ===\n');  
  
  // Example 1: Computer Boot  
  print('--- Computer Boot Example ---');  
  
  var computer = ComputerFacade();  
  computer.start();  
  print('');  
  computer.shutdown();  
}
```

```

print('');

// Example 2: Home Theater
print('--- Home Theater Example ---');

var homeTheater = HomeTheaterFacade();
homeTheater.watchMovie('The Matrix');
homeTheater.endMovie();
homeTheater.listenToRadio(98.7);
homeTheater.turnOffRadio();

print('');

// Example 3: Order Processing
print('--- Order Processing Example ---');

var orderSystem = OrderFacade();
orderSystem.placeOrder('Laptop', 1, 999.99, '123 Main St',
'customer@email.com');
orderSystem.cancelOrder('Laptop', 1, 999.99, 'customer@email.com');
}

```

Flyweight

Description: Uses **sharing to efficiently support a large number of fine-grained objects**. It separates intrinsic (shared) state from extrinsic (unique) state.

- **Why it is used:** To reduce memory usage when a large number of similar objects need to be created.
- **Where it is commonly used:** Text editors (reusing character objects), game engines (reusing tile or bullet objects), rendering systems.

Code Example (DART)

```

/*
 * FLYWEIGHT DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Flyweight pattern lets you fit more objects into the available RAM
by sharing common parts of state between multiple objects.
 * It separates intrinsic state (shared) from extrinsic state (unique).
 *
 * WHY DOES IT EXIST?
 * - When you need to reduce memory usage by sharing objects
 * - When a large number of similar objects should be shared
 * - When most of an object's state can be made extrinsic
 *
 * WHEN TO USE IT?
 * - When your application uses a large number of objects
 * - When storage costs are high because of the quantity of objects

```

```

* - When most of the object's state can be made extrinsic
* - When many groups of objects can be replaced by relatively few shared
objects
* - Common use cases: text editors (character formatting), game objects
(trees, soldiers), GUI elements (icons, fonts)
*
* PROS:
* - Reduces memory consumption by sharing objects
* - Improves performance when many similar objects are needed
* - Centralized state management
* - Can be combined with other patterns (Composite, Strategy)
*
* CONS:
* - May increase runtime complexity due to sharing
* - Requires careful identification of intrinsic vs extrinsic state
* - Can make code more complex
* - May sacrifice some encapsulation (extrinsic state passed to flyweight)
*
* REAL-WORLD ANALOGY:
* Think of a coffee shop. They don't make a new cup for every customer -
they reuse cups (intrinsic state: the cup itself). What changes is the
coffee inside (extrinsic state: the type of coffee). The cup is shared
(flyweight), while the coffee content varies per customer.
*/

```

```

// Flyweight Interface - defines the operations that can be performed on
flyweight objects

```

```

abstract class TreeType {
    void draw(int x, int y);
    String get name;
    String get color;
}

```

```

// Concrete Flyweight - implements the flyweight interface and stores
intrinsic state

```

```

class ConcreteTreeType extends TreeType {
    @override
    final String name;
    @override
    final String color;
    final String texture;

    ConcreteTreeType(this.name, this.color, this.texture);

    @override
    void draw(int x, int y) {
        print('Drawing $name tree at ($x, $y) with color $color and texture
$texture');
    }
}

```

```

// Flyweight Factory - creates and manages flyweight objects

```

```

class TreeFactory {
    final Map<String, TreeType> _treeTypes = {};
}

```

```
// Helper function to create a unique key for the map
String _getKey(String name, String color, String texture) {
    return '$name|$color|$texture';
}

TreeType getTreeType(String name, String color, String texture) {
    var key = _getKey(name, color, texture);

    if (!_treeTypes.containsKey(key)) {
        print('Creating new tree type: $name');
        _treeTypes[key] = ConcreteTreeType(name, color, texture);
    } else {
        print('Reusing existing tree type: $name');
    }

    return _treeTypes[key]!;
}

int getTotalTreeTypes() {
    return _treeTypes.length;
}

// Context - contains extrinsic state and uses flyweight objects
class Tree {
    final int x;
    final int y;
    final TreeType treeType;

    Tree(this.x, this.y, this.treeType);

    void draw() {
        treeType.draw(x, y);
    }
}

// Example 2: Character Formatting in Text Editor
// Shows how Flyweight can be used for text rendering

abstract class CharacterFormat {
    void apply(String text);
    String get font;
    int get size;
    String get color;
}

class ConcreteCharacterFormat extends CharacterFormat {
    @override
    final String font;
    @override
    final int size;
    @override
    final String color;
}
```

```

    final bool bold;
    final bool italic;

    ConcreteCharacterFormat(this.font, this.size, this.color, this.bold,
this.italic);

    @override
    void apply(String text) {
        var output = 'Rendering \'$text\' with font: $font, size: $size, color:
$color';
        if (bold) output += ', bold';
        if (italic) output += ', italic';
        print(output);
    }
}

class CharacterFormatFactory {
    final Map<String, CharacterFormat> _formats = {};

    String _getKey(String font, int size, String color, bool bold, bool
italic) {
        return '$font|$size|$color|${bold ? 'B' : ''}|${italic ? 'I' : ''}';
    }

    CharacterFormat getFormat(String font, int size, String color, bool bold,
bool italic) {
        var key = _getKey(font, size, color, bold, italic);

        if (!_formats.containsKey(key)) {
            print('Creating new format: $font ${size}pt');
            _formats[key] = ConcreteCharacterFormat(font, size, color, bold,
italic);
        } else {
            print('Reusing existing format: $font ${size}pt');
        }

        return _formats[key]!;
    }

    int getTotalFormats() {
        return _formats.length;
    }
}

class Character {
    final String symbol;
    final CharacterFormat format;

    Character(this.symbol, this.format);

    void render() {
        format.apply(symbol);
    }
}

```

```
// Example 3: Game Units
// Shows how Flyweight can be used for game objects

abstract class SoldierType {
  void render(int x, int y, String weapon);
  String get uniform;
  int get speed;
}

class ConcreteSoldierType extends SoldierType {
  @override
  final String uniform;
  @override
  final int speed;
  final String rank;

  ConcreteSoldierType(this.uniform, this.speed, this.rank);

  @override
  void render(int x, int y, String weapon) {
    print('Rendering $rank soldier at ($x, $y) with $uniform uniform, speed
    $speed, holding $weapon');
  }
}

class SoldierTypeFactory {
  final Map<String, SoldierType> _soldierTypes = {};

  String _getKey(String uniform, int speed, String rank) {
    return '$uniform|$speed|$rank';
  }

  SoldierType getSoldierType(String uniform, int speed, String rank) {
    var key = _getKey(uniform, speed, rank);

    if (!_soldierTypes.containsKey(key)) {
      print('Creating new soldier type: $rank');
      _soldierTypes[key] = ConcreteSoldierType(uniform, speed, rank);
    } else {
      print('Reusing existing soldier type: $rank');
    }

    return _soldierTypes[key]!;
  }

  int getTotalSoldierTypes() {
    return _soldierTypes.length;
  }
}

class Soldier {
  int x, y;
  final String weapon;
```

```
final SoldierType soldierType;

Soldier(this.x, this.y, this.weapon, this.soldierType);

void render() {
  soldierType.render(x, y, weapon);
}

void move(int newX, int newY) {
  x = newX;
  y = newY;
}
}

// Demo code
void main() {
  print('=== Flyweight Pattern Demo ===\n');

  // Example 1: Forest Trees
  print('--- Forest Trees Example ---');

  var treeFactory = TreeFactory();

  // Create trees with shared types
  var forest = <Tree>[];

  forest.add(Tree(10, 20, treeFactory.getTreeType('Oak', 'Green',
'Rough')));
  forest.add(Tree(30, 40, treeFactory.getTreeType('Oak', 'Green',
'Rough')));
  forest.add(Tree(50, 60, treeFactory.getTreeType('Oak', 'Green',
'Rough')));

  forest.add(Tree(15, 25, treeFactory.getTreeType('Pine', 'Dark Green',
'Smooth')));
  forest.add(Tree(35, 45, treeFactory.getTreeType('Pine', 'Dark Green',
'Smooth')));

  forest.add(Tree(20, 30, treeFactory.getTreeType('Oak', 'Green',
'Rough'))); // Reusing Oak

  print('\nDrawing forest:');
  for (var tree in forest) {
    tree.draw();
  }

  print('\nTotal unique tree types: ${treeFactory.getTotalTreeTypes()}');
  print('Total trees in forest: ${forest.length}');

  print('');

  // Example 2: Text Editor Characters
  print('--- Text Editor Characters Example ---');
```

```

var formatFactory = CharacterFormatFactory();

var document = <Character>[];

document.add(Character('H', formatFactory.getFormat('Arial', 12, 'Black',
true, false)));
document.add(Character('e', formatFactory.getFormat('Arial', 12, 'Black',
false, false)));
document.add(Character('l', formatFactory.getFormat('Arial', 12, 'Black',
false, false)));
document.add(Character('l', formatFactory.getFormat('Arial', 12, 'Black',
false, false)));
document.add(Character('o', formatFactory.getFormat('Arial', 12, 'Black',
false, false)));

document.add(Character(' ', formatFactory.getFormat('Arial', 12, 'Black',
false, false)));

document.add(Character('W', formatFactory.getFormat('Times New Roman',
14, 'Blue', true, true)));
document.add(Character('o', formatFactory.getFormat('Times New Roman',
14, 'Blue', false, true)));
document.add(Character('r', formatFactory.getFormat('Times New Roman',
14, 'Blue', false, true)));
document.add(Character('l', formatFactory.getFormat('Times New Roman',
14, 'Blue', false, true)));
document.add(Character('d', formatFactory.getFormat('Times New Roman',
14, 'Blue', false, true)));

print('\nRendering document:');
for (var character in document) {
  character.render();
}

print('\nTotal unique formats: ${formatFactory.getTotalFormats()}');
print('Total characters: ${document.length}');

print('');

// Example 3: Game Soldiers
print('--- Game Soldiers Example ---');

var soldierFactory = SoldierTypeFactory();

var army = <Soldier>[];

army.add(Soldier(10, 10, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));
army.add(Soldier(20, 20, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));
army.add(Soldier(30, 30, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));

army.add(Soldier(15, 15, 'Sniper',

```

```

soldierFactory.getSoldierType('Desert', 4, 'Sergeant')));
  army.add(Soldier(25, 25, 'Sniper',
soldierFactory.getSoldierType('Desert', 4, 'Sergeant')));

  army.add(Soldier(40, 40, 'Machine Gun',
soldierFactory.getSoldierType('Camouflage', 5, 'Private'))); // Reusing

  print('\nRendering army:');
  for (var soldier in army) {
    soldier.render();
  }

  print('\nTotal unique soldier types:
${soldierFactory.getTotalSoldierTypes()}');
  print('Total soldiers: ${army.length}');
}

```

Proxy

Description: Provides a **substitute or placeholder** for another object to control access to it.

- **Why it is used:** To add a layer of control over the original object — for access control, lazy initialization, logging, or caching — without changing the original object.
- **Where it is commonly used:** Virtual proxies (lazy loading images), security proxies (access control), remote proxies (network calls).

Code Example (DART)

```

/*
 * PROXY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Proxy pattern provides a surrogate or placeholder object to control
access to another object.
 * It acts as an intermediary between the client and the real object.
 *
 * WHY DOES IT EXIST?
 * - When you need to control access to an object
 * - When you want to add additional functionality when accessing an object
 * - When you need to defer the creation or initialization of expensive
objects
 *
 * WHEN TO USE IT?
 * - When you want to add a layer of security to access an object
 * - When you need to cache results of expensive operations
 * - When you want to delay object creation until it's actually needed
(lazy initialization)
 * - When you need to log or monitor access to an object
 * - Common use cases: remote objects, virtual proxies, protection proxies,
smart references

```

```

*
* PROS:
* - Can control access to the real object
* - Can add additional functionality without changing the real object
* - Can optimize performance (lazy loading, caching)
* - Can provide security by controlling access
*
* CONS:
* - Adds an extra layer of indirection
* - Can increase response time due to proxy overhead
* - Can make the code more complex
* - May introduce bugs if the proxy doesn't perfectly mimic the real
object
*
* REAL-WORLD ANALOGY:
* Think of a credit card. The credit card (proxy) represents your bank
account (real object). You don't carry your actual bank account around -
you use the card as a proxy. The card controls access to your money, adds
security (PIN, chip), and logs transactions, all while giving you access to
your funds.
*/

// Subject Interface - defines the common interface for RealSubject and
Proxy
abstract class Image {
  void display();
}

// RealSubject - the actual object that the proxy represents
class RealImage extends Image {
  final String filename;

  RealImage(this.filename) {
    _loadFromDisk();
  }

  // Helper method to load the image from disk
  void _loadFromDisk() {
    print('Loading $filename from disk...');
  }

  @override
  void display() {
    print('Displaying $filename');
  }
}

// Proxy - controls access to the RealSubject
class ProxyImage extends Image {
  final String filename;
  RealImage? _realImage;

  ProxyImage(this.filename);

```

```

@override
void display() {
  // Lazy initialization: only create the real image when needed
  if (_realImage == null) {
    print('Creating real image on first access...');
    _realImage = RealImage(filename);
  }
  _realImage!.display();
}
}

// Example 2: Protection Proxy
// Shows how Proxy can control access based on permissions

abstract class Database {
  void query(String sql);
}

class RealDatabase extends Database {
  @override
  void query(String sql) {
    print('Executing query: $sql');
  }
}

class DatabaseProxy extends Database {
  RealDatabase? _realDatabase;
  String _currentUser;
  final Map<String, bool> _permissions = {};

  DatabaseProxy(this._currentUser) {
    // Set up permissions (in real app, this would come from auth system)
    _permissions['admin'] = true;
    _permissions['user'] = false;
    _permissions['guest'] = false;
  }

  bool _hasPermission() {
    return _permissions[_currentUser] ?? false;
  }

  @override
  void query(String sql) {
    if (_hasPermission()) {
      _realDatabase ??= RealDatabase();
      _realDatabase!.query(sql);
    } else {
      print('Access denied: User \'$_currentUser\' does not have permission
to execute queries');
    }
  }

  void setUser(String user) {
    _currentUser = user;
  }
}

```

```
}  
}  
  
// Example 3: Caching Proxy  
// Shows how Proxy can cache expensive operations  
  
abstract class Video {  
  void play();  
}  
  
class RealVideo extends Video {  
  final String filename;  
  
  RealVideo(this.filename) {  
    _loadVideo();  
  }  
  
  void _loadVideo() {  
    print('Loading video $filename from server...');  
  }  
  
  @override  
  void play() {  
    print('Playing video: $filename');  
  }  
}  
  
class VideoProxy extends Video {  
  final String filename;  
  RealVideo? _realVideo;  
  bool _isLoading = false;  
  
  VideoProxy(this.filename);  
  
  @override  
  void play() {  
    if (!_isLoading) {  
      print('Loading video on first play...');  
      _realVideo = RealVideo(filename);  
      _isLoading = true;  
    } else {  
      print('Using cached video...');  
    }  
    _realVideo!.play();  
  }  
}  
  
// Example 4: Logging Proxy  
// Shows how Proxy can add logging functionality  
  
abstract class Service {  
  void doWork();  
}
```

```
class RealService extends Service {
  @override
  void doWork() {
    print('Service: Performing work...');
  }
}

class LoggingProxy extends Service {
  final RealService _realService = RealService();

  void _log(String message) {
    print('[LOG] $message');
  }

  @override
  void doWork() {
    _log('Before doWork');
    _realService.doWork();
    _log('After doWork');
  }
}

// Example 5: Remote Proxy (simulated)
// Shows how Proxy can represent remote objects

abstract class RemoteServer {
  void request(String data);
}

class RealRemoteServer extends RemoteServer {
  @override
  void request(String data) {
    print('Remote server processing request: $data');
  }
}

class RemoteServerProxy extends RemoteServer {
  RealRemoteServer? _remoteServer;
  final String serverAddress;

  RemoteServerProxy(this.serverAddress);

  void _connectToServer() {
    print('Connecting to remote server at $serverAddress...');
  }

  @override
  void request(String data) {
    if (_remoteServer == null) {
      _connectToServer();
      _remoteServer = RealRemoteServer();
    }
    print('Proxy forwarding request to remote server...');
    _remoteServer!.request(data);
  }
}
```

```
}  
}  
  
// Demo code  
void main() {  
  print('=== Proxy Pattern Demo ===\n');  
  
  // Example 1: Virtual Proxy (Lazy Loading)  
  print('--- Virtual Proxy (Lazy Loading) Example ---');  
  
  var image1 = ProxyImage('photo1.jpg');  
  var image2 = ProxyImage('photo2.jpg');  
  var image3 = ProxyImage('photo1.jpg'); // Same as image1  
  
  print('\nFirst display of image1:');  
  image1.display();  
  
  print('\nSecond display of image1 (should use cached):');  
  image1.display();  
  
  print('\nDisplay of image2:');  
  image2.display();  
  
  print('\nDisplay of image3 (same as image1, should use cached):');  
  image3.display();  
  
  print('');  
  
  // Example 2: Protection Proxy  
  print('--- Protection Proxy Example ---');  
  
  var db = DatabaseProxy('guest');  
  print('\nGuest trying to execute query:');  
  db.query('SELECT * FROM users');  
  
  print('\nSwitching to admin user:');  
  db.setUser('admin');  
  db.query('SELECT * FROM users');  
  
  print('\nSwitching back to regular user:');  
  db.setUser('user');  
  db.query('SELECT * FROM users');  
  
  print('');  
  
  // Example 3: Caching Proxy  
  print('--- Caching Proxy Example ---');  
  
  var video = VideoProxy('movie.mp4');  
  
  print('\nFirst play (will load):');  
  video.play();  
  
  print('\nSecond play (will use cache):');
```

```
video.play();

print('\nThird play (will use cache):');
video.play();

print('');

// Example 4: Logging Proxy
print('--- Logging Proxy Example ---');

var loggingService = LoggingProxy();
loggingService.doWork();

print('');

// Example 5: Remote Proxy
print('--- Remote Proxy Example ---');

var remoteProxy = RemoteServerProxy('192.168.1.100');

print('\nFirst request (will connect):');
remoteProxy.request('GET /api/data');

print('\nSecond request (already connected):');
remoteProxy.request('POST /api/data');
}
```

3. Behavioral Design Patterns

Overview

Behavioral patterns deal with **how objects interact and communicate** with each other. They focus on the assignment of responsibilities between objects and define clear communication patterns to make the system more flexible in carrying out behavior.

Chain of Responsibility

Description: Passes a request along a **chain of handlers**, where each handler decides to process the request or pass it to the next handler in the chain.

- **Why it is used:** To decouple the sender of a request from its receivers and give multiple objects the chance to handle the request.
 - **Where it is commonly used:** Middleware pipelines (HTTP request handling), event handling systems, logging with multiple levels.
-

Code Example (DART)

```

/*
 * CHAIN OF RESPONSIBILITY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Chain of Responsibility pattern lets you pass requests along a chain
of handlers.
 * Each handler decides either to process the request or to pass it to the
next handler in the chain.
 *
 * WHY DOES IT EXIST?
 * - When you want to decouple senders of a request from its receivers
 * - When multiple objects can handle a request without the sender knowing
which one will
 * - When you want to dynamically add or remove handlers at runtime
 *
 * WHEN TO USE IT?
 * - When more than one object can handle a request, and the handler isn't
known a priori
 * - When you want to issue a request to one of several objects without
specifying the receiver explicitly
 * - When the set of objects that can handle a request should be specified
dynamically
 * - Common use cases: event handling systems, logging frameworks, approval
workflows, help systems
 *
 * PROS:
 * - Decouples sender from receiver (sender doesn't know which handler will
process)
 * - Open/Closed Principle (can add new handlers without changing existing
code)
 * - Can control the order of handling
 * - Simplifies object interactions
 *
 * CONS:
 * - No guarantee that a request will be handled
 * - Can lead to deep chains that are hard to debug
 * - May affect performance if the chain is long
 * - Can be difficult to observe runtime characteristics
 *
 * REAL-WORLD ANALOGY:
 * Think of a customer service call center. When you call, your request
goes through a chain of representatives. If the first representative can't
help, they transfer you to a specialist. If that specialist can't help,
they transfer you to a manager. The call moves up the chain until someone
can help or you reach the end.
 */

// Handler interface - declares the handling method and sets a reference to
the next handler
abstract class Handler {
    Handler? _nextHandler;

```

```
// Set the next handler in the chain
Handler setNext(Handler handler) {
  _nextHandler = handler;
  return handler;
}

// Handle the request - to be implemented by concrete handlers
void handle(String request);

// Pass the request to the next handler if it exists
void passRequest(String request) {
  _nextHandler?.handle(request);
}

// Concrete Handler 1: Support Agent
class SupportAgent extends Handler {
  @override
  void handle(String request) {
    if (request == 'technical_issue' || request == 'basic_question') {
      print('SupportAgent: Handling request - $request');
    } else {
      print('SupportAgent: Passing to next handler...');
      passRequest(request);
    }
  }
}

// Concrete Handler 2: Technical Specialist
class TechnicalSpecialist extends Handler {
  @override
  void handle(String request) {
    if (request == 'complex_technical' || request == 'bug_report') {
      print('TechnicalSpecialist: Handling request - $request');
    } else {
      print('TechnicalSpecialist: Passing to next handler...');
      passRequest(request);
    }
  }
}

// Concrete Handler 3: Manager
class Manager extends Handler {
  @override
  void handle(String request) {
    if (request == 'refund' || request == 'complaint' || request ==
'escalation') {
      print('Manager: Handling request - $request');
    } else {
      print('Manager: Passing to next handler...');
      passRequest(request);
    }
  }
}
```

```
// Example 2: Document Approval Chain
// Shows how Chain of Responsibility can be used for approval workflows

abstract class Approver {
  Approver? _nextApprover;

  Approver setNext(Approver approver) {
    _nextApprover = approver;
    return approver;
  }

  void approveRequest(double amount);

  void passRequest(double amount) {
    _nextApprover?.approveRequest(amount);
  }
}

class TeamLead extends Approver {
  @override
  void approveRequest(double amount) {
    if (amount <= 1000) {
      print('TeamLead: Approved request of \${amount}');
    } else {
      print('TeamLead: Amount exceeds limit, passing to manager...');
      passRequest(amount);
    }
  }
}

class ManagerApprover extends Approver {
  @override
  void approveRequest(double amount) {
    if (amount <= 5000) {
      print('Manager: Approved request of \${amount}');
    } else {
      print('Manager: Amount exceeds limit, passing to director...');
      passRequest(amount);
    }
  }
}

class Director extends Approver {
  @override
  void approveRequest(double amount) {
    if (amount <= 20000) {
      print('Director: Approved request of \${amount}');
    } else {
      print('Director: Amount exceeds limit, passing to CEO...');
      passRequest(amount);
    }
  }
}
```

```
class CEO extends Approver {
  @override
  void approveRequest(double amount) {
    print('CEO: Approved request of \${amount} (final approver)');
  }
}

// Example 3: Logging Chain
// Shows how Chain of Responsibility can be used for logging

abstract class Logger {
  Logger? _nextLogger;

  Logger setNext(Logger logger) {
    _nextLogger = logger;
    return logger;
  }

  void log(String message, int severity);

  void passLog(String message, int severity) {
    _nextLogger?.log(message, severity);
  }
}

class ConsoleLogger extends Logger {
  @override
  void log(String message, int severity) {
    if (severity >= 1) {
      print('[CONSOLE] $message');
    }
    passLog(message, severity);
  }
}

class FileLogger extends Logger {
  @override
  void log(String message, int severity) {
    if (severity >= 2) {
      print('[FILE] $message');
    }
    passLog(message, severity);
  }
}

class ErrorLogger extends Logger {
  @override
  void log(String message, int severity) {
    if (severity >= 3) {
      print('[ERROR] $message');
    }
    passLog(message, severity);
  }
}
```

```
}

// Demo code
void main() {
    print('=== Chain of Responsibility Pattern Demo ===\n');

    // Example 1: Customer Service Chain
    print('--- Customer Service Chain ---');

    var agent = SupportAgent();
    var specialist = TechnicalSpecialist();
    var manager = Manager();

    agent.setNext(specialist).setNext(manager);

    print('\nRequest: basic_question');
    agent.handle('basic_question');

    print('\nRequest: complex_technical');
    agent.handle('complex_technical');

    print('\nRequest: refund');
    agent.handle('refund');

    print('\nRequest: unknown');
    agent.handle('unknown');

    print('');

    // Example 2: Document Approval Chain
    print('--- Document Approval Chain ---');

    var teamLead = TeamLead();
    var manager2 = ManagerApprover();
    var director = Director();
    var ceo = CEO();

    teamLead.setNext(manager2).setNext(director).setNext(ceo);

    print('\nRequest: $500');
    teamLead.approveRequest(500);

    print('\nRequest: $3000');
    teamLead.approveRequest(3000);

    print('\nRequest: $15000');
    teamLead.approveRequest(15000);

    print('\nRequest: $50000');
    teamLead.approveRequest(50000);

    print('');

    // Example 3: Logging Chain
```

```

print('--- Logging Chain ---');

var consoleLogger = ConsoleLogger();
var fileLogger = FileLogger();
var errorLogger = ErrorLogger();

consoleLogger.setNext(fileLogger).setNext(errorLogger);

print('\nLog: Info message (severity 1)');
consoleLogger.log('Info message', 1);

print('\nLog: Warning message (severity 2)');
consoleLogger.log('Warning message', 2);

print('\nLog: Error message (severity 3)');
consoleLogger.log('Error message', 3);
}

```

Command

Description: Encapsulates a request as an object, allowing you to parameterize clients with different requests, queue operations, and support undoable actions.

- **Why it is used:** To decouple the object that sends a request from the object that executes it, enabling flexible command management.
- **Where it is commonly used:** Undo/redo systems, task schedulers, GUI buttons and menu actions, transactional operations.

Code Example (DART)

```

/*
 * COMMAND DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Command pattern turns a request into a stand-alone object that
 * contains all information about the request.
 * It lets you parameterize clients with different requests, queue or log
 * requests, and support undoable operations.
 *
 * WHY DOES IT EXIST?
 * - When you want to parameterize objects with operations
 * - When you want to queue operations, schedule their execution, or
 * execute them remotely
 * - When you need to support undo operations
 * - When you want to decouple the object that invokes an operation from
 * the one that knows how to perform it
 *
 * WHEN TO USE IT?
 * - When you want to parameterize objects with an operation to execute
 * - When you want to queue operations, schedule their execution, or

```

```

execute them remotely
* - When you need to support undo/redo functionality
* - When you want to decouple the invoker from the receiver
* - Common use cases: GUI buttons and menus, multi-level undo, macro
recording, transactional systems
*
* PROS:
* - Decouples the object that invokes the operation from the one that
knows how to perform it
* - Can add new commands without changing existing code (Open/Closed
Principle)
* - Allows you to assemble commands into more complex commands (composite
pattern)
* - Supports undo/redo operations
*
* CONS:
* - Can lead to many small command classes (code complexity)
* - Can complicate the code if the receiver and command are tightly
coupled
* - May introduce additional layers of indirection
* - Can make debugging more difficult
*
* REAL-WORLD ANALOGY:
* Think of a restaurant order. The customer (client) places an order
(command) with the waiter (invoker). The waiter writes down the order and
gives it to the kitchen (receiver). The kitchen executes the command
(prepare the meal). The order can be cancelled (undo) before it's
prepared, and multiple orders can be queued.
*/

// Receiver - knows how to perform the operations
class Light {
  void on() {
    print('Light is ON');
  }

  void off() {
    print('Light is OFF');
  }
}

class Stereo {
  void on() {
    print('Stereo is ON');
  }

  void off() {
    print('Stereo is OFF');
  }

  void setVolume(int level) {
    print('Stereo volume set to $level');
  }
}

```

```
// Command interface - declares the execute method
abstract class Command {
    void execute();
    void undo();
}

// Concrete Command 1: Light On
class LightOnCommand extends Command {
    final Light _light;

    LightOnCommand(this._light);

    @override
    void execute() {
        _light.on();
    }

    @override
    void undo() {
        _light.off();
    }
}

// Concrete Command 2: Light Off
class LightOffCommand extends Command {
    final Light _light;

    LightOffCommand(this._light);

    @override
    void execute() {
        _light.off();
    }

    @override
    void undo() {
        _light.on();
    }
}

// Concrete Command 3: Stereo On with Volume
class StereoOnCommand extends Command {
    final Stereo _stereo;
    int _previousVolume = 0;

    StereoOnCommand(this._stereo);

    @override
    void execute() {
        _stereo.on();
        _stereo.setVolume(10);
        _previousVolume = 10;
    }
}
```

```

    @override
    void undo() {
      _stereo.setVolume(_previousVolume);
      _stereo.off();
    }
  }

// Concrete Command 4: Stereo Off
class StereoOffCommand extends Command {
  final Stereo _stereo;

  StereoOffCommand(this._stereo);

  @override
  void execute() {
    _stereo.off();
  }

  @override
  void undo() {
    _stereo.on();
  }
}

// Invoker - asks the command to carry out the request
class RemoteControl {
  final List<Command> _commandHistory = [];
  final List<Command> _undoStack = [];

  void setCommand(Command command) {
    _commandHistory.add(command);
  }

  void buttonPressed(int slot) {
    if (slot < _commandHistory.length) {
      _commandHistory[slot].execute();
      _undoStack.add(_commandHistory[slot]);
    }
  }

  void undoButtonPressed() {
    if (_undoStack.isNotEmpty) {
      final command = _undoStack.removeLast();
      command.undo();
    } else {
      print('Nothing to undo');
    }
  }
}

// Example 2: Macro Command
// Shows how multiple commands can be combined into one

```

```
class MacroCommand extends Command {
  final List<Command> _commands = [];

  void addCommand(Command command) {
    _commands.add(command);
  }

  @override
  void execute() {
    for (final command in _commands) {
      command.execute();
    }
  }

  @override
  void undo() {
    // Undo in reverse order
    for (int i = _commands.length - 1; i >= 0; i--) {
      _commands[i].undo();
    }
  }
}

// Example 3: Transaction Command
// Shows how Command can be used for transactional operations

class BankAccount {
  double _balance = 0;

  BankAccount([double initialBalance = 0]) : _balance = initialBalance;

  void deposit(double amount) {
    _balance += amount;
    print('Deposited \$$amount. New balance: \$$_balance');
  }

  void withdraw(double amount) {
    if (_balance >= amount) {
      _balance -= amount;
      print('Withdrew \$$amount. New balance: \$$_balance');
    } else {
      print('Insufficient funds');
    }
  }

  double getBalance() => _balance;
}

class DepositCommand extends Command {
  final BankAccount _account;
  final double _amount;

  DepositCommand(this._account, this._amount);
```

```
@override
void execute() {
  _account.deposit(_amount);
}

@override
void undo() {
  _account.withdraw(_amount);
}
}

class WithdrawCommand extends Command {
  final BankAccount _account;
  final double _amount;
  bool _success = false;

  WithdrawCommand(this._account, this._amount);

  @override
  void execute() {
    final oldBalance = _account.getBalance();
    _account.withdraw(_amount);
    _success = _account.getBalance() < oldBalance;
  }

  @override
  void undo() {
    if (_success) {
      _account.deposit(_amount);
    }
  }
}

// Demo code
void main() {
  print('=== Command Pattern Demo ===\n');

  // Example 1: Remote Control
  print('--- Remote Control Example ---');

  final livingRoomLight = Light();
  final livingRoomStereo = Stereo();

  final lightOn = LightOnCommand(livingRoomLight);
  final lightOff = LightOffCommand(livingRoomLight);
  final stereoOn = StereoOnCommand(livingRoomStereo);
  final stereoOff = StereoOffCommand(livingRoomStereo);

  final remote = RemoteControl();
  remote.setCommand(lightOn);
  remote.setCommand(lightOff);
  remote.setCommand(stereoOn);
  remote.setCommand(stereoOff);
}
```

```
print('\nTurning on light (slot 0):');
remote.buttonPressed(0);

print('\nTurning on stereo (slot 2):');
remote.buttonPressed(2);

print('\nUndo last action:');
remote.undoButtonPressed();

print('\nTurning off light (slot 1):');
remote.buttonPressed(1);

print('\nUndo last action:');
remote.undoButtonPressed();

print('');

// Example 2: Macro Command
print('--- Macro Command Example ---');

final partyMode = MacroCommand();
partyMode.addCommand(lightOn);
partyMode.addCommand(stereoOn);

final shutdownMode = MacroCommand();
shutdownMode.addCommand(stereoOff);
shutdownMode.addCommand(lightOff);

print('\nActivating party mode:');
partyMode.execute();

print('\nActivating shutdown mode:');
shutdownMode.execute();

print('\nUndo shutdown:');
shutdownMode.undo();

print('');

// Example 3: Transaction Command
print('--- Transaction Command Example ---');

final myAccount = BankAccount(1000);

print('\nInitial balance: \${myAccount.getBalance()}');

final deposit = DepositCommand(myAccount, 500);
final withdraw = WithdrawCommand(myAccount, 200);

print('\nExecuting deposit:');
deposit.execute();

print('\nExecuting withdrawal:');
withdraw.execute();
```

```

print('\nUndo withdrawal:');
withdraw.undo();

print('\nFinal balance: \${myAccount.getBalance()}');
}

```

Iterator

Description: Provides a way to **sequentially access elements** of a collection without exposing its underlying representation.

- **Why it is used:** To traverse different types of collections in a uniform way without depending on their internal structure.
- **Where it is commonly used:** Iterating over lists, trees, graphs, and custom data structures in any programming language.

Code Example (DART)

```

/*
 * ITERATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Iterator pattern provides a way to access the elements of an
 * aggregate object sequentially without exposing its underlying
 * representation.
 * It traverses the elements of a collection without knowing its internal
 * structure.
 *
 * WHY DOES IT EXIST?
 * - When you want to access a collection's contents without exposing its
 * internal representation
 * - When you want to provide multiple ways to traverse a collection
 * - When you want to provide a uniform interface for traversing different
 * data structures
 *
 * WHEN TO USE IT?
 * - When you have a collection of objects and want to traverse it without
 * exposing its internal structure
 * - When you want to provide multiple traversal algorithms for the same
 * collection
 * - When you want to simplify the interface for accessing a complex data
 * structure
 * - Common use cases: collections, tree structures, database result sets,
 * file systems
 *
 * PROS:
 * - Single Responsibility Principle (separates traversal logic from the
 * collection)
 * - Open/Closed Principle (can add new iterators without changing the

```

```

collection)
* - Provides a uniform interface for traversing different collections
* - Can support multiple traversals simultaneously
*
* CONS:
* - Can be overkill for simple collections
* - May violate encapsulation if the iterator exposes too much about the
collection
* - Can be less efficient than direct access
* - May complicate the code for simple use cases
*
* REAL-WORLD ANALOGY:
* Think of a TV remote control with channel up/down buttons. You don't
need to know how the TV stores channels - you just use the next/previous
buttons to navigate. The remote (iterator) provides a simple interface to
traverse the channels (collection) without knowing the internal channel
storage mechanism.
*/

// Iterator interface - declares the operations for traversing
abstract class Iterator<T> {
  bool hasNext();
  T next();
}

// Aggregate interface - declares the method for creating an iterator
abstract class Aggregate<T> {
  Iterator<T> createIterator();
}

// Concrete Aggregate - implements the aggregate interface
class BookCollection implements Aggregate<String> {
  final List<String> _items = [];

  void add(String item) {
    _items.add(item);
  }

  @override
  Iterator<String> createIterator() {
    return BookIterator(this);
  }

  int size() => _items.length;

  String get(int index) => _items[index];
}

// Concrete Iterator
class BookIterator implements Iterator<String> {
  final BookCollection _collection;
  int _position = 0;

  BookIterator(this._collection);

```

```

@override
bool hasNext() {
    return _position < _collection.size();
}

@override
String next() {
    if (hasNext()) {
        return _collection.get(_position++);
    }
    return '';
}
}

// Example 2: Tree Traversal Iterator
// Shows how Iterator can be used for tree structures

class TreeNode<T> {
    T data;
    TreeNode<T>? left;
    TreeNode<T>? right;

    TreeNode(this.data);
}

class BinaryTreeIterator<T> implements Iterator<T> {
    final List<TreeNode<T>> _stack = [];

    BinaryTreeIterator(TreeNode<T>? root) {
        _pushLeft(root);
    }

    void _pushLeft(TreeNode<T>? node) {
        while (node != null) {
            _stack.add(node);
            node = node.left;
        }
    }

    @override
    bool hasNext() {
        return _stack.isNotEmpty;
    }

    @override
    T next() {
        if (!hasNext()) return null as T;

        final node = _stack.removeLast();

        if (node.right != null) {
            _pushLeft(node.right);
        }
    }
}

```

```

        return node.data;
    }
}

// Example 3: Custom Container with Iterator
// Shows how to implement iterator for a custom data structure

class CustomList<T> implements Aggregate<T> {
    _Node<T>? _head;

    void add(T item) {
        final newNode = _Node(item);
        if (_head == null) {
            _head = newNode;
        } else {
            _Node<T>? current = _head;
            while (current!.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
    }

    @override
    Iterator<T> createIterator() {
        return ListIterator(_head);
    }
}

class _Node<T> {
    T data;
    _Node<T>? next;

    _Node(this.data);
}

class ListIterator<T> implements Iterator<T> {
    _Node<T>? _current;

    ListIterator(this._current);

    @override
    bool hasNext() {
        return _current != null;
    }

    @override
    T next() {
        if (!hasNext()) return null as T;
        final data = _current!.data;
        _current = _current!.next;
        return data;
    }
}

```

```
}

// Demo code
void main() {
    print('=== Iterator Pattern Demo ===\n');

    // Example 1: Book Collection
    print('--- Book Collection Example ---');

    final books = BookCollection();
    books.add('Design Patterns');
    books.add('Clean Code');
    books.add('Refactoring');
    books.add('The Pragmatic Programmer');

    final bookIterator = books.createIterator();

    print('\nIterating through books:');
    while (bookIterator.hasNext()) {
        print('  - ${bookIterator.next()}');
    }

    print('');

    // Example 2: Binary Tree Traversal
    print('--- Binary Tree Traversal Example ---');

    // Build a simple binary tree
    final root = TreeNode(5);
    root.left = TreeNode(3);
    root.right = TreeNode(7);
    root.left!.left = TreeNode(1);
    root.left!.right = TreeNode(4);
    root.right!.left = TreeNode(6);
    root.right!.right = TreeNode(9);

    final treeIterator = BinaryTreeIterator(root);

    print('\nIn-order traversal:');
    while (treeIterator.hasNext()) {
        print('  ${treeIterator.next()}');
    }

    print('');

    // Example 3: Custom List Iterator
    print('--- Custom List Iterator Example ---');

    final numbers = CustomList<int>();
    numbers.add(10);
    numbers.add(20);
    numbers.add(30);
    numbers.add(40);
    numbers.add(50);
}
```

```

    final listIterator = numbers.createIterator();

    print('\nIterating through custom list:');
    while (listIterator.hasNext()) {
        print('    ${listIterator.next()}');
    }
}

```

Mediator

Description: Defines an object that **encapsulates how a set of objects interact**. It promotes loose coupling by preventing objects from referring to each other directly.

- **Why it is used:** To reduce the chaotic dependencies between many objects communicating directly. All communication goes through the mediator.
- **Where it is commonly used:** Chat room systems, air traffic control systems, UI dialog form coordination.

Code Example (DART)

```

/*
 * MEDIATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Mediator pattern defines an object that encapsulates how a set of
objects interact.
 * It promotes loose coupling by preventing objects from referring to each
other explicitly.
 *
 * WHY DOES IT EXIST?
 * - When you want to reduce the chaotic dependencies between objects
 * - When you want to centralize complex communications and control logic
between related objects
 * - When you want to reuse the interaction logic without duplicating it
across multiple objects
 *
 * WHEN TO USE IT?
 * - When a set of objects communicate in well-defined but complex ways
 * - When you want to customize a behavior that's distributed between
several objects without subclassing
 * - When you have many objects that need to interact with each other
 * - Common use cases: chat rooms, air traffic control, GUI frameworks,
auction systems
 *
 * PROS:
 * - Single Responsibility Principle (extracts communication logic to a
separate class)
 * - Open/Closed Principle (can add new colleagues without changing the
mediator)

```

```

* - Reduces coupling between colleagues
* - Simplifies object interactions
*
* CONS:
* - The mediator can become a monolith (god object) if not carefully
designed
* - Can make the system harder to understand due to indirect communication
* - May reduce performance due to indirection
* - Can be overkill for simple interactions
*
* REAL-WORLD ANALOGY:
* Think of an air traffic control tower. Planes (colleagues) don't
communicate directly with each other - they all communicate through the
tower (mediator). The tower coordinates takeoffs, landings, and routes,
ensuring planes don't collide. This centralizes the coordination logic and
prevents planes from needing to know about each other.
*/

// Mediator interface - declares the communication interface
abstract class Mediator {
  void sendMessage(String message, String sender);
}

// Colleague interface - declares the interface for communicating with the
mediator
abstract class Colleague {
  final Mediator _mediator;
  final String _name;

  Colleague(this._mediator, this._name);

  void send(String message) {
    _mediator.sendMessage(message, _name);
  }

  void receive(String message);

  String get name => _name;
}

// Concrete Colleague 1
class User extends Colleague {
  User(super.mediator, super.name);

  @override
  void receive(String message) {
    print('$_name received: $message');
  }
}

// Concrete Colleague 2
class Bot extends Colleague {
  Bot(super.mediator, super.name);
}

```

```
@override
void receive(String message) {
  print('$_name received: $message');
  // Bot can auto-reply
  send('Auto-reply: Thanks for your message!');
}

// Concrete Mediator
class ChatRoom implements Mediator {
  final Map<String, Colleague> _colleagues = {};

  void addUser(Colleague colleague) {
    _colleagues[colleague.name] = colleague;
  }

  @override
  void sendMessage(String message, String sender) {
    print('$sender sends: $message');
    for (final entry in _colleagues.entries) {
      if (entry.key != sender) {
        entry.value.receive(message);
      }
    }
  }
}

// Example 2: Air Traffic Control
// Shows how Mediator can coordinate complex interactions

class Aircraft {
  final String _flightNumber;
  int _altitude = 0;

  Aircraft(this._flightNumber);

  void requestTakeoff() {
    print('$_flightNumber requesting takeoff clearance...');
    // In real implementation, would send to mediator
  }

  void requestLanding() {
    print('$_flightNumber requesting landing clearance...');
    // In real implementation, would send to mediator
  }

  void setAltitude(int alt) {
    _altitude = alt;
    print('$_flightNumber at altitude $_altitude feet');
  }

  String get flightNumber => _flightNumber;
}
```

```
class ControlTower implements Mediator {
  final Map<String, Aircraft> _aircrafts = {};

  void registerAircraft(Aircraft aircraft) {
    _aircrafts[aircraft.flightNumber] = aircraft;
  }

  @override
  void sendMessage(String message, String sender) {
    // Simplified for demo
    print('Control Tower: $message from $sender');
  }

  void grantTakeoff(String flightNumber) {
    print('Control Tower: Takeoff granted to $flightNumber');
    final aircraft = _aircrafts[flightNumber];
    if (aircraft != null) {
      aircraft.setAltitude(10000);
    }
  }

  void grantLanding(String flightNumber) {
    print('Control Tower: Landing granted to $flightNumber');
    final aircraft = _aircrafts[flightNumber];
    if (aircraft != null) {
      aircraft.setAltitude(0);
    }
  }
}

// Example 3: Smart Home System
// Shows how Mediator can coordinate home automation

class SmartDevice {
  final Mediator _homeHub;
  final String _deviceName;
  bool _isOn = false;

  SmartDevice(this._homeHub, this._deviceName);

  void turnOn() {
    _isOn = true;
    print('$_deviceName turned ON');
    _homeHub.sendMessage('$_deviceName turned ON', _deviceName);
  }

  void turnOff() {
    _isOn = false;
    print('$_deviceName turned OFF');
    _homeHub.sendMessage('$_deviceName turned OFF', _deviceName);
  }

  String get name => _deviceName;
}
```

```
bool get isOn => _isOn;
}

class SmartLight extends SmartDevice {
  SmartLight(super.homeHub, super.name);
}

class SmartThermostat extends SmartDevice {
  int _temperature = 70;

  SmartThermostat(super.homeHub, super.name);

  void setTemperature(int temp) {
    _temperature = temp;
    print('$name set to $_temperature°F');
  }

  int get temperature => _temperature;
}

class HomeHub implements Mediator {
  final Map<String, SmartDevice> _devices = {};

  void addDevice(SmartDevice device) {
    _devices[device.name] = device;
  }

  @override
  void sendMessage(String message, String sender) {
    print('Home Hub: $message');

    // Example logic: if thermostat turns on, turn on lights
    if (sender.contains('Thermostat') && message.contains('turned ON')) {
      for (final device in _devices.values) {
        if (device.name.contains('Light') && !device.isOn) {
          device.turnOn();
        }
      }
    }
  }

  void activateAwayMode() {
    print('Home Hub: Activating away mode...');
    for (final device in _devices.values) {
      if (device.isOn) {
        device.turnOff();
      }
    }
  }

  void activateHomeMode() {
    print('Home Hub: Activating home mode...');
    for (final device in _devices.values) {
      if (device.name.contains('Light')) {

```

```
        device.turnOn();
    }
}
}

// Demo code
void main() {
    print('=== Mediator Pattern Demo ===\n');

    // Example 1: Chat Room
    print('--- Chat Room Example ---');

    final chatRoom = ChatRoom();

    final user1 = User(chatRoom, 'Alice');
    final user2 = User(chatRoom, 'Bob');
    final bot = Bot(chatRoom, 'HelperBot');

    chatRoom.addUser(user1);
    chatRoom.addUser(user2);
    chatRoom.addUser(bot);

    print('\nAlice sends a message:');
    user1.send('Hello everyone!');

    print('\nBob sends a message:');
    user2.send('Hi Alice!');

    print('');

    // Example 2: Air Traffic Control
    print('--- Air Traffic Control Example ---');

    final controlTower = ControlTower();

    final flight1 = Aircraft('AA123');
    final flight2 = Aircraft('UA456');

    controlTower.registerAircraft(flight1);
    controlTower.registerAircraft(flight2);

    print('\nFlight AA123 requests takeoff:');
    controlTower.grantTakeoff('AA123');

    print('\nFlight UA456 requests landing:');
    controlTower.grantLanding('UA456');

    print('');

    // Example 3: Smart Home System
    print('--- Smart Home System Example ---');

    final homeHub = HomeHub();
```

```

final livingRoomLight = SmartLight(homeHub, 'Living Room Light');
final bedroomLight = SmartLight(homeHub, 'Bedroom Light');
final thermostat = SmartThermostat(homeHub, 'Main Thermostat');

homeHub.addDevice(livingRoomLight);
homeHub.addDevice(bedroomLight);
homeHub.addDevice(thermostat);

print('\nTurning on thermostat (should trigger lights:');
thermostat.turnOn();

print('\nActivating away mode:');
homeHub.activateAwayMode();

print('\nActivating home mode:');
homeHub.activateHomeMode();
}

```

Memento

Description: Captures and externalizes an object's **internal state** so it can be restored later, without violating encapsulation.

- **Why it is used:** To implement undo/redo functionality by saving snapshots of an object's state.
- **Where it is commonly used:** Text editors (undo history), game save states, transactional systems.

Code Example (DART)

```

/*
 * MEMENTO DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Memento pattern lets you save and restore the previous state of an
 * object without revealing the details of its implementation.
 * It provides the ability to restore an object to its previous state
 * (undo).
 *
 * WHY DOES IT EXIST?
 * - When you need to implement undo/redo functionality
 * - When you want to create snapshots of an object's state
 * - When direct access to the object's internal representation would
 * violate encapsulation
 *
 * WHEN TO USE IT?
 * - When you need to save and restore the state of an object without
 * exposing its internal structure
 * - When you want to implement undo/redo operations
 * - When you need to maintain a history of states
 * - Common use cases: text editors (undo/redo), game states, database

```

```

transactions, configuration management
*
* PROS:
* - Preserves encapsulation (doesn't expose internal state)
* - Simplifies the originator (doesn't need to manage state history)
* - Can provide snapshots at different times
* - Supports undo/redo functionality
*
* CONS:
* - Can be expensive if the originator has large state
* - May consume significant memory if many mementos are stored
* - Caretaker must track the originator's lifecycle
* - Can be complex to implement for complex objects
*
* REAL-WORLD ANALOGY:
* Think of a game save system. When you save a game, the game creates a
snapshot of your current state (position, health, inventory). This snapshot
(memento) is stored separately. Later, you can restore the game to that
exact state by loading the snapshot. The game doesn't need to know how the
snapshot is stored, just that it can be restored.
*/

// Memento - stores the internal state of the Originator
class TextEditorMemento {
  final String content;
  final String timestamp;

  TextEditorMemento(this.content) : timestamp = DateTime.now().toString();

  String get getContent => content;
  String get getTimestamp => timestamp;
}

// Originator - creates a memento containing a snapshot of its current
state
class TextEditor {
  String _content = '';

  void write(String text) {
    _content += text;
    print('Text: "$text" added');
  }

  void setContent(String newContent) {
    _content = newContent;
  }

  String get content => _content;

  TextEditorMemento save() {
    print('Saving current state...');
    return TextEditorMemento(_content);
  }
}

```

```
void restore(TextEditorMemento memento) {
  _content = memento.getContent();
  print('Restored state from ${memento.getTimestamp}');
  print('Current content: "$_content"');
}
}

// Caretaker - manages mementos, never operates on their contents
class History {
  final List<TextEditorMemento> _mementos = [];

  void push(TextEditorMemento memento) {
    _mementos.add(memento);
  }

  TextEditorMemento? pop() {
    if (_mementos.isEmpty) {
      return null;
    }
    return _mementos.removeLast();
  }

  int get size => _mementos.length;
}

// Example 2: Game State Memento
// Shows how Memento can be used for game saves

class GameStateMemento {
  final int level;
  final int health;
  final int score;
  final String position;

  GameStateMemento(this.level, this.health, this.score, this.position);

  int get getLevel => level;
  int get getHealth => health;
  int get getScore => score;
  String get getPosition => position;
}

class GameCharacter {
  int _level = 1;
  int _health = 100;
  int _score = 0;
  String _position = 'Start';

  void moveTo(String newPosition) {
    _position = newPosition;
    print('Moved to $_position');
  }

  void takeDamage(int damage) {
```

```

    _health -= damage;
    if (_health < 0) _health = 0;
    print('Took $damage damage. Health: $_health');
  }

void gainScore(int points) {
  _score += points;
  print('Gained $points points. Score: $_score');
}

void levelUp() {
  _level++;
  _health = 100;
  print('Level up! Now at level $_level');
}

void displayStatus() {
  print('=== Character Status ===');
  print('Level: $_level');
  print('Health: $_health');
  print('Score: $_score');
  print('Position: $_position');
  print('=====');
}

GameStateMemento saveState() {
  print('Saving game state...');
  return GameStateMemento(_level, _health, _score, _position);
}

void restoreState(GameStateMemento memento) {
  _level = memento.getLevel();
  _health = memento.getHealth();
  _score = memento.getScore();
  _position = memento.getPosition();
  print('Game state restored!');
  displayStatus();
}
}

class GameSaveManager {
  final List<GameStateMemento> _saves = [];

  void createSave(GameStateMemento save) {
    _saves.add(save);
    print('Save created. Total saves: ${_saves.length}');
  }

  GameStateMemento? loadSave(int index) {
    if (index >= 0 && index < _saves.length) {
      print('Loading save ${index + 1}...');
      return _saves[index];
    }
    return null;
  }
}

```

```
}

int get saveCount => _saves.length;
}

// Example 3: Configuration Memento
// Shows how Memento can be used for configuration management

class ConfigurationMemento {
  final String theme;
  final int fontSize;
  final bool notificationsEnabled;

  ConfigurationMemento(this.theme, this.fontSize,
    this.notificationsEnabled);

  String get getTheme => theme;
  int get getFontSize => fontSize;
  bool get getNotificationsEnabled => notificationsEnabled;
}

class ApplicationSettings {
  String _theme = 'light';
  int _fontSize = 12;
  bool _notificationsEnabled = true;

  void setTheme(String newTheme) {
    _theme = newTheme;
    print('Theme changed to $_theme');
  }

  void setFontSize(int size) {
    _fontSize = size;
    print('Font size set to $_fontSize');
  }

  void setNotificationsEnabled(bool enabled) {
    _notificationsEnabled = enabled;
    print('Notifications ${enabled ? "enabled" : "disabled"}');
  }

  void displaySettings() {
    print('=== Current Settings ===');
    print('Theme: $_theme');
    print('Font Size: $_fontSize');
    print('Notifications: ${_notificationsEnabled ? "Enabled" :
"Disabled"}');
    print('=====');
  }

  ConfigurationMemento saveConfiguration() {
    return ConfigurationMemento(_theme, _fontSize, _notificationsEnabled);
  }
}
```

```
void restoreConfiguration(ConfigurationMemento memento) {
    _theme = memento.getTheme();
    _fontSize = memento.getFontSize();
    _notificationsEnabled = memento.getNotificationsEnabled();
    print('Configuration restored!');
    displaySettings();
}

}

// Demo code
void main() {
    print('=== Memento Pattern Demo ===\n');

    // Example 1: Text Editor with Undo
    print('--- Text Editor with Undo ---');

    final editor = TextEditor();
    final history = History();

    editor.write('Hello ');
    history.push(editor.save());

    editor.write('World ');
    history.push(editor.save());

    editor.write('!');
    history.push(editor.save());

    print('\nCurrent content: "${editor.content}");
    print('History size: ${history.size}');

    print('\nPerforming undo...');
    final memento1 = history.pop();
    if (memento1 != null) {
        editor.restore(memento1);
    }

    print('\nPerforming undo...');
    final memento2 = history.pop();
    if (memento2 != null) {
        editor.restore(memento2);
    }

    print('');

    // Example 2: Game Save System
    print('--- Game Save System ---');

    final player = GameCharacter();
    final saveManager = GameSaveManager();

    print('\nInitial state:');
    player.displayStatus();
}
```

```
print('\nPlaying game...');
player.moveTo('Forest');
player.gainScore(100);
player.takeDamage(10);

print('\nCreating save 1...');
saveManager.createSave(player.saveState());

print('\nContinuing game...');
player.moveTo('Castle');
player.levelUp();
player.gainScore(500);
player.takeDamage(30);

print('\nCreating save 2...');
saveManager.createSave(player.saveState());

print('\nCurrent state:');
player.displayStatus();

print('\nLoading save 1...');
final save1 = saveManager.loadSave(0);
if (save1 != null) {
  player.restoreState(save1);
}

print('');

// Example 3: Configuration Management
print('--- Configuration Management Example ---');

final settings = ApplicationSettings();

print('\nDefault settings:');
settings.displaySettings();

final originalConfig = settings.saveConfiguration();

print('\nChanging settings...');
settings.setTheme('dark');
settings.setFontSize(14);
settings.setNotificationsEnabled(false);

print('\nNew settings:');
settings.displaySettings();

print('\nRestoring original settings...');
settings.restoreConfiguration(originalConfig);
}
```

Observer

Description: Defines a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.

- **Why it is used:** To implement event-driven systems where multiple parts of an application need to react to changes in a shared object.
- **Where it is commonly used:** Event listeners, pub/sub systems, real-time dashboards, MVC architecture (model notifying views).

Code Example (DART)

```
/*
 * OBSERVER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Observer pattern defines a subscription mechanism to notify multiple
objects about any events that happen to the object they are observing.
 * It establishes a one-to-many dependency between objects so that when one
object changes state, all its dependents are notified.
 *
 * WHY DOES IT EXIST?
 * - When you need to notify multiple objects about changes to a single
object
 * - When you want to decouple the subject from its observers
 * - When you want to broadcast changes to multiple subscribers dynamically
 *
 * WHEN TO USE IT?
 * - When changes to one object require changing other objects
 * - When the set of objects that need to be notified is unknown or dynamic
 * - When you want to implement a publish-subscribe model
 * - Common use cases: GUI event handling, news feeds, social media
notifications, stock price updates
 *
 * PROS:
 * - Open/Closed Principle (can add new observers without changing the
subject)
 * - Establishes loose coupling between subject and observers
 * - Can broadcast communications to multiple objects
 * - Supports dynamic subscription/unsubscription
 *
 * CONS:
 * - Can lead to unexpected updates if observers are not careful
 * - Can cause performance issues with many observers
 * - Debugging can be difficult due to indirect calls
 * - May cause memory leaks if observers don't unsubscribe
 *
 * REAL-WORLD ANALOGY:
 * Think of a YouTube channel (subject) and subscribers (observers). When
the channel uploads a new video, all subscribers get notified. Subscribers
can subscribe or unsubscribe at any time. The channel doesn't need to know
who is subscribed - it just broadcasts notifications to all current
subscribers.
```

```
*/

// Observer interface - declares the update method
abstract class Observer {
  void update(String message);
}

// Subject interface - declares methods for attaching, detaching, and
notifying observers
abstract class Subject {
  final List<Observer> _observers = [];

  void attach(Observer observer) {
    _observers.add(observer);
    print('Attached an observer');
  }

  void detach(Observer observer) {
    _observers.remove(observer);
    print('Detached an observer');
  }

  void notify(String message) {
    print('Notifying observers...');
    for (final observer in _observers) {
      observer.update(message);
    }
  }
}

// Concrete Subject
class YouTubeChannel extends Subject {
  final String _channelName;

  YouTubeChannel(this._channelName);

  void uploadVideo(String videoTitle) {
    print('\n$_channelName uploaded: $videoTitle');
    notify('New video: $videoTitle');
  }

  void goLive(String streamTitle) {
    print('\n$_channelName is live: $streamTitle');
    notify('Live stream started: $streamTitle');
  }
}

// Concrete Observer 1
class Subscriber implements Observer {
  final String _name;

  Subscriber(this._name);

  @override
```

```

    void update(String message) {
        print('  $_name received notification: $message');
    }
}

// Concrete Observer 2
class NotificationService implements Observer {
    @override
    void update(String message) {
        print('  [NotificationService] Push notification sent: $message');
    }
}

// Example 2: Stock Market
// Shows how Observer can be used for real-time updates

abstract class StockObserver {
    void onStockPriceChange(String symbol, double price);
}

class StockMarket {
    final List<StockObserver> _observers = [];
    final Map<String, double> _stockPrices = {};

    void addObserver(StockObserver observer) {
        _observers.add(observer);
    }

    void removeObserver(StockObserver observer) {
        _observers.remove(observer);
    }

    void updateStockPrice(String symbol, double price) {
        _stockPrices[symbol] = price;
        print('\n$symbol price updated to \$$price');
        _notifyObservers(symbol, price);
    }

    void _notifyObservers(String symbol, double price) {
        for (final observer in _observers) {
            observer.onStockPriceChange(symbol, price);
        }
    }
}

class Investor implements StockObserver {
    final String _name;
    final Map<String, int> _portfolio = {};

    Investor(this._name);

    void buyStock(String symbol, int shares) {
        _portfolio[symbol] = (_portfolio[symbol] ?? 0) + shares;
        print('$_name bought $shares shares of $symbol');
    }
}

```

```

    }

    @override
    void onStockPriceChange(String symbol, double price) {
        final shares = _portfolio[symbol];
        if (shares != null && shares > 0) {
            print('  $_name owns $shares shares of $symbol (current price:
\\$price)');
        }
    }
}

class TradingBot implements StockObserver {
    @override
    void onStockPriceChange(String symbol, double price) {
        print('  [TradingBot] Analyzing $symbol at \\$price');
        if (price > 100) {
            print('  [TradingBot] Recommendation: SELL');
        } else if (price < 50) {
            print('  [TradingBot] Recommendation: BUY');
        }
    }
}

// Example 3: Weather Station
// Shows how Observer can be used for sensor data

abstract class WeatherObserver {
    void onWeatherUpdate(double temperature, double humidity, double
pressure);
}

class WeatherStation {
    final List<WeatherObserver> _observers = [];
    double _temperature = 0;
    double _humidity = 0;
    double _pressure = 0;

    void addObserver(WeatherObserver observer) {
        _observers.add(observer);
    }

    void removeObserver(WeatherObserver observer) {
        _observers.remove(observer);
    }

    void setMeasurements(double temp, double hum, double press) {
        _temperature = temp;
        _humidity = hum;
        _pressure = press;
        print('\\nWeather updated: $_temperature°C, $_humidity% humidity,
$_pressure hPa');
        _notifyObservers();
    }
}

```

```
void _notifyObservers() {
  for (final observer in _observers) {
    observer.onWeatherUpdate(_temperature, _humidity, _pressure);
  }
}

class DisplayDevice implements WeatherObserver {
  final String _deviceName;

  DisplayDevice(this._deviceName);

  @override
  void onWeatherUpdate(double temperature, double humidity, double
pressure) {
    print('  [$_deviceName] Displaying: $temperature°C, $humidity%,
$pressure hPa');
  }
}

class AlertSystem implements WeatherObserver {
  @override
  void onWeatherUpdate(double temperature, double humidity, double
pressure) {
    print('  [AlertSystem] Checking conditions...');
    if (temperature > 35) {
      print('  [AlertSystem] WARNING: High temperature!');
    }
    if (humidity > 80) {
      print('  [AlertSystem] WARNING: High humidity!');
    }
    if (pressure < 980) {
      print('  [AlertSystem] WARNING: Low pressure (storm coming)!');
    }
  }
}

// Demo code
void main() {
  print('=== Observer Pattern Demo ===\n');

  // Example 1: YouTube Channel
  print('--- YouTube Channel Example ---');

  final channel = YouTubeChannel('Tech Channel');

  final subscriber1 = Subscriber('Alice');
  final subscriber2 = Subscriber('Bob');
  final notificationService = NotificationService();

  channel.attach(subscriber1);
  channel.attach(subscriber2);
  channel.attach(notificationService);
}
```

```
channel.uploadVideo('Design Patterns Tutorial');

channel.detach(subscriber2);

channel.goLive('Live Q&A Session');

print('');

// Example 2: Stock Market
print('--- Stock Market Example ---');

final market = StockMarket();

final investor1 = Investor('John');
final investor2 = Investor('Jane');
final tradingBot = TradingBot();

market.addObserver(investor1);
market.addObserver(investor2);
market.addObserver(tradingBot);

investor1.buyStock('AAPL', 100);
investor2.buyStock('GOOGL', 50);

market.updateStockPrice('AAPL', 150.50);
market.updateStockPrice('GOOGL', 45.00);
market.updateStockPrice('TSLA', 200.00);

print('');

// Example 3: Weather Station
print('--- Weather Station Example ---');

final station = WeatherStation();

final phoneDisplay = DisplayDevice('Phone');
final watchDisplay = DisplayDevice('Smart Watch');
final alertSystem = AlertSystem();

station.addObserver(phoneDisplay);
station.addObserver(watchDisplay);
station.addObserver(alertSystem);

station.setMeasurements(25.0, 60.0, 1013.0);
station.setMeasurements(38.0, 85.0, 975.0);
station.setMeasurements(20.0, 50.0, 1020.0);
}
```

State

Description: Allows an object to **alter its behavior when its internal state changes**. The object will appear to change its class.

- **Why it is used:** To replace complex conditional statements (`if/switch`) that depend on the object's state with dedicated state classes.
- **Where it is commonly used:** Traffic light systems, vending machines, order status workflows, game character states.

Code Example (DART)

```
/*
 * STATE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The State pattern allows an object to alter its behavior when its
internal state changes.
 * The object appears to change its class by delegating state-specific
behavior to different state objects.
 *
 * WHY DOES IT EXIST?
 * - When an object's behavior depends on its state and must change at
runtime
 * - When you have large conditional statements that switch between states
 * - When you want to avoid monolithic conditional logic
 *
 * WHEN TO USE IT?
 * - When an object has many states and its behavior changes based on those
states
 * - When you have large conditional operators that switch between states
 * - When you want to eliminate conditional logic from the main class
 * - Common use cases: game characters, vending machines, audio/video
players, document editors
 *
 * PROS:
 * - Single Responsibility Principle (each state is a separate class)
 * - Open/Closed Principle (can add new states without changing existing
code)
 * - Eliminates conditional logic from the context
 * - Makes state transitions explicit
 *
 * CONS:
 * - Can increase the number of classes in the system
 * - State transitions can be complex to manage
 * - May make the code more complex for simple scenarios
 * - Can lead to tight coupling between states
 *
 * REAL-WORLD ANALOGY:
 * Think of a vending machine. It has different states: "No Coin", "Has
Coin", "Dispensing", "Out of Stock". When you insert a coin, it transitions
from "No Coin" to "Has Coin". When you press a button, it transitions to
"Dispensing". The machine's behavior changes based on its current state,
```

but the machine itself doesn't change - it just delegates to the appropriate state object.

```
*/

// State interface - declares the state-specific methods
abstract class State {
  void insertCoin();
  void ejectCoin();
  void pressButton();
  void dispense();
}

// Context - maintains a reference to the current state
class VendingMachine {
  late State _currentState;
  int _coinCount = 0;
  int _stockCount;

  VendingMachine(this._stockCount) {
    // Initial state depends on stock
    if (_stockCount > 0) {
      _currentState = NoCoinState(this);
    } else {
      _currentState = SoldOutState(this);
    }
  }

  void setState(State state) {
    _currentState = state;
  }

  void insertCoin() {
    _currentState.insertCoin();
  }

  void ejectCoin() {
    _currentState.ejectCoin();
  }

  void pressButton() {
    _currentState.pressButton();
  }

  void dispense() {
    _currentState.dispense();
  }

  void addCoin() {
    _coinCount++;
    print('Coin inserted. Total coins: $_coinCount');
  }

  void returnCoin() {
    if (_coinCount > 0) {
```

```

        _coinCount--;
        print('Coin returned. Total coins: $_coinCount');
    }
}

void releaseProduct() {
    if (_stockCount > 0) {
        _stockCount--;
        _coinCount--;
        print('Product dispensed. Stock: $_stockCount, Coins: $_coinCount');
    }
}

int get coinCount => _coinCount;
int get stockCount => _stockCount;
}

// Concrete State 1: No Coin
class NoCoinState implements State {
    final VendingMachine _machine;

    NoCoinState(this._machine);

    @override
    void insertCoin() {
        print('Coin inserted...');
        _machine.addCoin();
        _machine.setState(HasCoinState(_machine));
    }

    @override
    void ejectCoin() {
        print('No coin to eject');
    }

    @override
    void pressButton() {
        print('Please insert a coin first');
    }

    @override
    void dispense() {
        print('Please insert a coin first');
    }
}

// Concrete State 2: Has Coin
class HasCoinState implements State {
    final VendingMachine _machine;

    HasCoinState(this._machine);

    @override
    void insertCoin() {

```

```
        print('Coin already inserted');
    }

    @override
    void ejectCoin() {
        print('Coin ejected');
        _machine.returnCoin();
        _machine.setState(NoCoinState(_machine));
    }

    @override
    void pressButton() {
        print('Button pressed...');
        if (_machine.stockCount > 0) {
            _machine.setState(DispensingState(_machine));
            _machine.dispense();
        } else {
            print('Out of stock');
            _machine.setState(SoldOutState(_machine));
        }
    }

    @override
    void dispense() {
        print('Press button to dispense');
    }
}

// Concrete State 3: Dispensing
class DispensingState implements State {
    final VendingMachine _machine;

    DispensingState(this._machine);

    @override
    void insertCoin() {
        print('Please wait, dispensing...');
    }

    @override
    void ejectCoin() {
        print('Cannot eject during dispensing');
    }

    @override
    void pressButton() {
        print('Already dispensing...');
    }

    @override
    void dispense() {
        _machine.releaseProduct();
        if (_machine.stockCount > 0) {
            _machine.setState(NoCoinState(_machine));
        }
    }
}
```

```
    } else {
        _machine.setState(SoldOutState(_machine));
    }
}

// Concrete State 4: Sold Out
class SoldOutState implements State {
    SoldOutState(VendingMachine machine);

    @override
    void insertCoin() {
        print('Cannot insert coin, machine is sold out');
    }

    @override
    void ejectCoin() {
        print('Cannot eject, no coin inserted');
    }

    @override
    void pressButton() {
        print('Machine is sold out');
    }

    @override
    void dispense() {
        print('Machine is sold out');
    }
}

// Example 2: Audio Player States
// Shows how State can be used for media players

abstract class AudioPlayerState {
    void clickPlay();
    void clickStop();
    void clickNext();
    void clickPrevious();
}

class AudioPlayer {
    late AudioPlayerState _currentState;
    bool _isPlaying = false;
    int _currentTrack = 1;
    int _totalTracks;

    AudioPlayer(this._totalTracks) {
        _currentState = StoppedState(this);
    }

    void setState(AudioPlayerState state) {
        _currentState = state;
    }
}
```

```
void clickPlay() {
  _currentState.clickPlay();
}

void clickStop() {
  _currentState.clickStop();
}

void clickNext() {
  _currentState.clickNext();
}

void clickPrevious() {
  _currentState.clickPrevious();
}

void startPlaying() {
  _isPlaying = true;
  print('Started playing track $_currentTrack');
}

void stopPlaying() {
  _isPlaying = false;
  print('Stopped playing');
}

void nextTrack() {
  _currentTrack = (_currentTrack % _totalTracks) + 1;
  print('Moved to track $_currentTrack');
}

void previousTrack() {
  _currentTrack = _currentTrack == 1 ? _totalTracks : _currentTrack - 1;
  print('Moved to track $_currentTrack');
}

bool get isPlaying => _isPlaying;
}

class PlayingState implements AudioPlayerState {
  final AudioPlayer _player;

  PlayingState(this._player);

  @override
  void clickPlay() {
    print('Already playing');
  }

  @override
  void clickStop() {
    print('Stopping playback...');
    _player.stopPlaying();
  }
}
```

```
    _player.setState(StoppedState(_player));
  }

  @override
  void clickNext() {
    print('Skipping to next track...');
    _player.nextTrack();
  }

  @override
  void clickPrevious() {
    print('Going to previous track...');
    _player.previousTrack();
  }
}

class StoppedState implements AudioPlayerState {
  final AudioPlayer _player;

  StoppedState(this._player);

  @override
  void clickPlay() {
    print('Starting playback...');
    _player.startPlaying();
    _player.setState(PlayingState(_player));
  }

  @override
  void clickStop() {
    print('Already stopped');
  }

  @override
  void clickNext() {
    print('Skipping to next track...');
    _player.nextTrack();
  }

  @override
  void clickPrevious() {
    print('Going to previous track...');
    _player.previousTrack();
  }
}

// Demo code
void main() {
  print('=== State Pattern Demo ===\n');

  // Example 1: Vending Machine
  print('--- Vending Machine Example ---');

  final machine = VendingMachine(5); // 5 items in stock
```

```
print('\nAttempt to press button without coin:');
machine.pressButton();

print('\nInsert coin:');
machine.insertCoin();

print('\nPress button:');
machine.pressButton();

print('\nInsert another coin:');
machine.insertCoin();

print('\nPress button:');
machine.pressButton();

print('\nEject coin:');
machine.ejectCoin();

print('');

// Example 2: Audio Player
print('--- Audio Player Example ---');

final player = AudioPlayer(10); // 10 tracks

print('\nClick play (stopped state:');
player.clickPlay();

print('\nClick next:');
player.clickNext();

print('\nClick stop:');
player.clickStop();

print('\nClick previous:');
player.clickPrevious();

print('\nClick play again:');
player.clickPlay();
}
```

Strategy

Description: Defines a **family of algorithms**, encapsulates each one, and makes them interchangeable. The strategy lets the algorithm vary independently from clients that use it.

- **Why it is used:** To select behavior at runtime without changing the client that uses it. Promotes the Open/Closed Principle.
- **Where it is commonly used:** Sorting algorithms (choose bubble sort vs. quicksort at runtime), payment processing systems, data compression strategies.

Code Example (DART)

```

/*
 * STRATEGY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Strategy pattern defines a family of algorithms, encapsulates each
one, and makes them interchangeable.
 * It lets the algorithm vary independently from clients that use it.
 *
 * WHY DOES IT EXIST?
 * - When you want to define a family of algorithms and make them
interchangeable
 * - When you want to avoid conditional statements for selecting algorithms
 * - When you want to separate algorithm implementation from the code that
uses it
 *
 * WHEN TO USE IT?
 * - When you have multiple ways to do an operation and want to switch
between them at runtime
 * - When you want to isolate the algorithm implementation from the client
code
 * - When you have complex conditional logic that selects different
algorithms
 * - Common use cases: sorting algorithms, payment processing, compression,
route planning, validation
 *
 * PROS:
 * - Open/Closed Principle (can add new strategies without changing
existing code)
 * - Separates algorithm implementation from the code that uses it
 * - Eliminates conditional statements for selecting algorithms
 * - Allows runtime selection of algorithms
 *
 * CONS:
 * - Clients must be aware of the differences between strategies
 * - Can increase the number of classes in the system
 * - May lead to performance overhead if strategies are complex
 * - Can make the code more complex for simple scenarios
 *
 * REAL-WORLD ANALOGY:
 * Think of a GPS navigation app. It can use different route calculation
strategies: fastest route, shortest distance, avoid highways, scenic route.
You can switch between these strategies at runtime without changing the app
itself. The app (context) delegates the route calculation to the selected
strategy.
 */

// Strategy interface - declares the algorithm method
abstract class PaymentStrategy {
    void pay(double amount);
}

```

```

// Concrete Strategy 1: Credit Card
class CreditCardPayment implements PaymentStrategy {
  final String _cardNumber;
  final String _expiryDate;

  CreditCardPayment(this._cardNumber, this._expiryDate);

  @override
  void pay(double amount) {
    print('Paying \$$amount with Credit Card ending in
    ${_cardNumber.substring(_cardNumber.length - 4)}');
    print('  Card: ****-****-
    ****-${_cardNumber.substring(_cardNumber.length - 4)}');
    print('  Expiry: $_expiryDate');
  }
}

// Concrete Strategy 2: PayPal
class PayPalPayment implements PaymentStrategy {
  final String _email;

  PayPalPayment(this._email);

  @override
  void pay(double amount) {
    print('Paying \$$amount with PayPal');
    print('  Account: $_email');
  }
}

// Concrete Strategy 3: Crypto
class CryptoPayment implements PaymentStrategy {
  final String _walletAddress;

  CryptoPayment(this._walletAddress);

  @override
  void pay(double amount) {
    print('Paying \$$amount with Cryptocurrency');
    print('  Wallet: ${_walletAddress.substring(0, 10)}...');
  }
}

// Context - uses the strategy
class ShoppingCart {
  PaymentStrategy? _paymentStrategy;
  final List<_Item> _items = [];

  void setPaymentStrategy(PaymentStrategy strategy) {
    _paymentStrategy = strategy;
  }

  void addItem(String item, double price) {

```

```

    _items.add(_Item(item, price));
  }

  double getTotal() {
    return _items.fold(0, (total, item) => total + item.price);
  }

  void checkout() {
    print('\n=== Checkout ===');
    print('Items:');
    for (final item in _items) {
      print('  - ${item.name}: \${item.price}');
    }
    print('Total: \${getTotal()}');

    if (_paymentStrategy != null) {
      _paymentStrategy!.pay(getTotal());
    } else {
      print('No payment method selected');
    }
  }
}

class _Item {
  final String name;
  final double price;

  _Item(this.name, this.price);
}

// Example 2: Sorting Strategies
// Shows how Strategy can be used for different algorithms

abstract class SortingStrategy {
  void sort(List<int> data);
}

class BubbleSort implements SortingStrategy {
  @override
  void sort(List<int> data) {
    print('Using Bubble Sort...');
    final n = data.length;
    for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
        if (data[j] > data[j + 1]) {
          final temp = data[j];
          data[j] = data[j + 1];
          data[j + 1] = temp;
        }
      }
    }
  }
}

```

```
class QuickSort implements SortingStrategy {
  @override
  void sort(List<int> data) {
    print('Using Quick Sort...');
    _quickSort(data, 0, data.length - 1);
  }

  void _quickSort(List<int> data, int low, int high) {
    if (low < high) {
      final pi = _partition(data, low, high);
      _quickSort(data, low, pi - 1);
      _quickSort(data, pi + 1, high);
    }
  }

  int _partition(List<int> data, int low, int high) {
    final pivot = data[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
      if (data[j] < pivot) {
        i++;
        final temp = data[i];
        data[i] = data[j];
        data[j] = temp;
      }
    }
    final temp = data[i + 1];
    data[i + 1] = data[high];
    data[high] = temp;
    return i + 1;
  }
}

class MergeSort implements SortingStrategy {
  @override
  void sort(List<int> data) {
    print('Using Merge Sort...');
    _mergeSort(data, 0, data.length - 1);
  }

  void _mergeSort(List<int> data, int left, int right) {
    if (left < right) {
      final mid = left + (right - left) ~/ 2;
      _mergeSort(data, left, mid);
      _mergeSort(data, mid + 1, right);
      _merge(data, left, mid, right);
    }
  }

  void _merge(List<int> data, int left, int mid, int right) {
    final n1 = mid - left + 1;
    final n2 = right - mid;

    final L = data.sublist(left, left + n1);
```

```

    final R = data.sublist(mid + 1, mid + 1 + n2);

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            data[k++] = L[i++];
        } else {
            data[k++] = R[j++];
        }
    }

    while (i < n1) data[k++] = L[i++];
    while (j < n2) data[k++] = R[j++];
}

class Sorter {
    SortingStrategy? _strategy;

    void setSortingStrategy(SortingStrategy strategy) {
        _strategy = strategy;
    }

    void sortData(List<int> data) {
        print('\nOriginal data: ${data.join(' ')}');

        if (_strategy != null) {
            _strategy!.sort(data);
        } else {
            print('No sorting strategy selected');
        }

        print('Sorted data: ${data.join(' ')}');
    }
}

// Example 3: Route Planning Strategies
// Shows how Strategy can be used for navigation

abstract class RouteStrategy {
    void calculateRoute(String from, String to);
}

class FastestRoute implements RouteStrategy {
    @override
    void calculateRoute(String from, String to) {
        print('Calculating fastest route from $from to $to');
        print('  Using highways and main roads to minimize time');
        print('  Estimated time: 25 minutes');
    }
}

class ShortestRoute implements RouteStrategy {
    @override

```

```
void calculateRoute(String from, String to) {
    print('Calculating shortest route from $from to $to');
    print('  Using direct path to minimize distance');
    print('  Estimated distance: 15 km');
}

class ScenicRoute implements RouteStrategy {
    @override
    void calculateRoute(String from, String to) {
        print('Calculating scenic route from $from to $to');
        print('  Using parks and landmarks for best views');
        print('  Estimated time: 45 minutes');
    }
}

class NavigationSystem {
    RouteStrategy? _strategy;

    void setRouteStrategy(RouteStrategy strategy) {
        _strategy = strategy;
    }

    void navigate(String from, String to) {
        print('\n=== Navigation ===');
        print('From: $from');
        print('To: $to');

        if (_strategy != null) {
            _strategy!.calculateRoute(from, to);
        } else {
            print('No route strategy selected');
        }
    }
}

// Demo code
void main() {
    print('=== Strategy Pattern Demo ===\n');

    // Example 1: Payment Processing
    print('--- Payment Processing Example ---');

    final cart = ShoppingCart();
    cart.addItem('Laptop', 999.99);
    cart.addItem('Mouse', 29.99);
    cart.addItem('Keyboard', 79.99);

    print('\nPaying with Credit Card:');
    cart.setPaymentStrategy(CreditCardPayment('4111111111111111', '12/25'));
    cart.checkout();

    print('\nPaying with PayPal:');
    cart.setPaymentStrategy(PayPalPayment('user@example.com'));
```

```

    cart.checkout();

    print('\nPaying with Crypto:');
    cart.setPaymentStrategy(CryptoPayment('0x1234567890abcdef'));
    cart.checkout();

    // Example 2: Sorting Algorithms
    print('\n--- Sorting Algorithms Example ---');

    final sorter = Sorter();
    final data = [64, 34, 25, 12, 22, 11, 90];

    sorter.setSortingStrategy(BubbleSort());
    sorter.sortData(List.from(data));

    sorter.setSortingStrategy(QuickSort());
    sorter.sortData(List.from(data));

    sorter.setSortingStrategy(MergeSort());
    sorter.sortData(List.from(data));

    // Example 3: Route Planning
    print('\n--- Route Planning Example ---');

    final nav = NavigationSystem();

    print('\nFastest route:');
    nav.setRouteStrategy(FastestRoute());
    nav.navigate('Home', 'Office');

    print('\nShortest route:');
    nav.setRouteStrategy(ShortestRoute());
    nav.navigate('Home', 'Office');

    print('\nScenic route:');
    nav.setRouteStrategy(ScenicRoute());
    nav.navigate('Home', 'Office');
}

```

Template Method

Description: Defines the **skeleton of an algorithm** in a base class, but lets subclasses override specific steps without changing the algorithm's overall structure.

- **Why it is used:** To avoid code duplication when multiple classes share the same algorithm structure but differ in specific steps.
- **Where it is commonly used:** Data parsing pipelines, report generation systems, game AI turn sequences.

Code Example (DART)

```

/*
 * TEMPLATE METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Template Method pattern defines the skeleton of an algorithm in the
base class but lets subclasses override specific steps without changing the
algorithm's structure.
 * It's a behavioral pattern that uses inheritance to define parts of an
algorithm.
 *
 * WHY DOES IT EXIST?
 * - When you have an algorithm with invariant parts and variable parts
 * - When you want to avoid code duplication by extracting common behavior
 * - When you want to control the extension points of an algorithm
 *
 * WHEN TO USE IT?
 * - When you have several classes that contain almost identical algorithms
with minor differences
 * - When you want to let subclasses extend only certain parts of a large
algorithm
 * - When you want to enforce the overall structure of an algorithm while
allowing customization
 * - Common use cases: data processing pipelines, frameworks, build
systems, report generation
 *
 * PROS:
 * - Reuses common code in the base class
 * - Follows Open/Closed Principle (extend behavior without modifying
existing code)
 * - Provides control over the extension points
 * - Reduces code duplication
 *
 * CONS:
 * - Can be difficult to maintain if the template method is complex
 * - May limit flexibility due to rigid structure
 * - Can lead to class explosion if many variations are needed
 * - Subclasses may need to implement methods they don't use
 *
 * REAL-WORLD ANALOGY:
 * Think of cooking a recipe. The recipe (template method) defines the
steps: prepare ingredients, cook, serve. The specific ingredients and
cooking methods can vary (subclass implementations), but the overall
structure remains the same. You can make pasta, curry, or stir-fry using
the same recipe structure with different implementations.
 */

// Abstract Class - defines the template method and abstract primitive
operations
abstract class DataProcessor {
    // Template method - defines the algorithm structure
    void processData() {
        print('\n=== Starting Data Processing ===');
    }
}

```

```
    readData();
    validateData();
    transformData();
    saveData();
    print('=== Data Processing Complete ===');
}

// Primitive operations - to be implemented by subclasses
void readData();
void validateData();
void transformData();
void saveData();

// Hook method - optional override point
void logProcessing() {
    print('Processing logged');
}

}

// Concrete Class 1: CSV Data Processor
class CSVDataProcessor extends DataProcessor {
    String _data = '';

    @override
    void readData() {
        print('Reading CSV file...');
        _data = 'name,age,city\nJohn,30,NYC\nJane,25,LA';
    }

    @override
    void validateData() {
        print('Validating CSV format...');
        print('CSV format is valid');
    }

    @override
    void transformData() {
        print('Transforming CSV to JSON...');
        // Simulate transformation
        _data = ' [{"name":"John","age":30,"city":"NYC"},
{"name":"Jane","age":25,"city":"LA"} ]';
    }

    @override
    void saveData() {
        print('Saving JSON file...');
        print('Data saved: $_data');
    }
}

// Concrete Class 2: XML Data Processor
class XMLDataProcessor extends DataProcessor {
    String _data = '';
```

```
@override
void readData() {
  print('Reading XML file...');
  _data = '<users><user><name>John</name><age>30</age></user></users>';
}

@override
void validateData() {
  print('Validating XML structure...');
  print('XML structure is valid');
}

@override
void transformData() {
  print('Transforming XML to CSV...');
  // Simulate transformation
  _data = 'name,age\nJohn,30';
}

@override
void saveData() {
  print('Saving CSV file...');
  print('Data saved: $_data');
}
}

// Example 2: Report Generation
// Shows how Template Method can be used for report generation

abstract class ReportGenerator {
  // Template method
  void generateReport() {
    print('\n=== Generating Report ===');
    collectData();
    analyzeData();
    formatReport();
    exportReport();
    print('=== Report Generated ===');
  }

  void collectData();
  void analyzeData();
  void formatReport();
  void exportReport();
}

class SalesReport extends ReportGenerator {
  @override
  void collectData() {
    print('Collecting sales data from database...');
  }

  @override
  void analyzeData() {
```

```

        print('Analyzing sales trends...');
        print('Sales increased by 15% this quarter');
    }

    @override
    void formatReport() {
        print('Formatting sales report with charts...');
    }

    @override
    void exportReport() {
        print('Exporting to PDF...');
    }
}

class InventoryReport extends ReportGenerator {
    @override
    void collectData() {
        print('Collecting inventory data...');
    }

    @override
    void analyzeData() {
        print('Analyzing stock levels...');
        print('3 items need restocking');
    }

    @override
    void formatReport() {
        print('Formatting inventory report with tables...');
    }

    @override
    void exportReport() {
        print('Exporting to Excel...');
    }
}

// Example 3: Game Character Creation
// Shows how Template Method can be used for character creation

abstract class GameCharacter {
    // Template method
    void createCharacter() {
        print('\n=== Creating Character ===');
        chooseName();
        selectRace();
        selectClass();
        assignStats();
        equipStartingItems();
        print('=== Character Created ===');
    }

    void chooseName();

```

```
void selectRace();
void selectClass();
void assignStats();
void equipStartingItems();

// Hook method
void displayCharacter() {
    print('Character ready for adventure!');
}

}

class Warrior extends GameCharacter {
    @override
    void chooseName() {
        print('Name chosen: Thorin');
    }

    @override
    void selectRace() {
        print('Race selected: Dwarf');
    }

    @override
    void selectClass() {
        print('Class selected: Warrior');
    }

    @override
    void assignStats() {
        print('Stats assigned: STR: 18, DEX: 12, CON: 16, INT: 8, WIS: 10');
    }

    @override
    void equipStartingItems() {
        print('Equipped: Iron Sword, Steel Shield, Chain Mail');
    }

    @override
    void displayCharacter() {
        super.displayCharacter();
        print('Special ability: Battle Rage');
    }
}

class Mage extends GameCharacter {
    @override
    void chooseName() {
        print('Name chosen: Gandalf');
    }

    @override
    void selectRace() {
        print('Race selected: Human');
    }
}
```

```
@override
void selectClass() {
  print('Class selected: Mage');
}

@override
void assignStats() {
  print('Stats assigned: STR: 8, DEX: 14, CON: 10, INT: 18, WIS: 16');
}

@override
void equipStartingItems() {
  print('Equipped: Wooden Staff, Robe, Spellbook');
}

@override
void displayCharacter() {
  super.displayCharacter();
  print('Special ability: Fireball');
}
}

class Rogue extends GameCharacter {
  @override
  void chooseName() {
    print('Name chosen: Legolas');
  }

  @override
  void selectRace() {
    print('Race selected: Elf');
  }

  @override
  void selectClass() {
    print('Class selected: Rogue');
  }

  @override
  void assignStats() {
    print('Stats assigned: STR: 12, DEX: 18, CON: 12, INT: 14, WIS: 12');
  }

  @override
  void equipStartingItems() {
    print('Equipped: Dagger, Leather Armor, Lockpicks');
  }

  @override
  void displayCharacter() {
    super.displayCharacter();
    print('Special ability: Stealth');
  }
}
```

```
}

// Demo code
void main() {
    print('=== Template Method Pattern Demo ===\n');

    // Example 1: Data Processing
    print('\n--- Data Processing Example ---');

    final csvProcessor = CSVDataProcessor();
    csvProcessor.processData();

    final xmlProcessor = XMLDataProcessor();
    xmlProcessor.processData();

    // Example 2: Report Generation
    print('\n--- Report Generation Example ---');

    final salesReport = SalesReport();
    salesReport.generateReport();

    final inventoryReport = InventoryReport();
    inventoryReport.generateReport();

    // Example 3: Game Character Creation
    print('\n--- Game Character Creation Example ---');

    final warrior = Warrior();
    warrior.createCharacter();

    final mage = Mage();
    mage.createCharacter();

    final rogue = Rogue();
    rogue.createCharacter();
}
```

Visitor

Description: Lets you **add new operations to existing object structures** without modifying the objects. A visitor object carries the operation to be performed.

- **Why it is used:** To separate an algorithm from the object structure it operates on, making it easy to add new operations without touching existing classes.
- **Where it is commonly used:** Compilers (AST traversal), document export systems (export to PDF, HTML, XML), tax/discount calculation systems.

Summary Table

Category	Focus	Goal
Creational	Object creation	Control how objects are instantiated
Structural	Object composition	Build flexible and efficient structures
Behavioral	Object interaction	Define clear communication between objects

Tip for Beginners: Don't try to memorize all patterns at once. Start with the most commonly used ones — **Singleton**, **Factory Method**, **Observer**, **Strategy**, and **Decorator** — and learn the rest as you encounter real-world problems they solve.

Code Example (DART)

```
/*
 * VISITOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Visitor pattern lets you define a new operation without changing the
classes of the elements on which it operates.
 * It separates the algorithm from the object structure it operates on.
 *
 * WHY DOES IT EXIST?
 * - When you need to add operations to a complex object structure without
changing the structure
 * - When you want to perform operations across different object types
 * - When you want to centralize related operations in a single class
 *
 * WHEN TO USE IT?
 * - When you have a complex object structure with different types of
elements
 * - When you need to perform many unrelated operations on these elements
 * - When the object structure is stable but you want to add new operations
frequently
 * - Common use cases: document processing, compiler AST traversal, file
system operations, shopping cart calculations
 *
 * PROS:
 * - Open/Closed Principle (can add new operations without changing element
classes)
 * - Single Responsibility Principle (moves related operations to a single
visitor)
 * - Can accumulate state while traversing the object structure
 * - Works well with Composite pattern
 *
 * CONS:
 * - Adding new element classes requires updating all visitors
 * - Can violate encapsulation by exposing element internals
 * - Can lead to complex visitor hierarchies
 * - May make the code harder to understand
 *
 * REAL-WORLD ANALOGY:
```

```
* Think of a tax inspector visiting different types of businesses. The
inspector (visitor) visits a restaurant, a retail store, and a factory
(elements). Each business type calculates taxes differently. The inspector
knows how to calculate taxes for each type but doesn't change how the
businesses operate. You can add new types of inspections (health inspector,
fire inspector) without changing the businesses.
*/

// Visitor interface - declares visit methods for each concrete element
abstract class Visitor {
  void visitBook(Book book);
  void visitFruit(Fruit fruit);
  void visitElectronic(Electronic electronic);
}

// Element interface - declares the accept method
abstract class Item {
  void accept(Visitor visitor);
  double getPrice();
}

// Concrete Element 1: Book
class Book extends Item {
  final String _title;
  final String _author;
  final double _price;

  Book(this._title, this._author, this._price);

  @override
  void accept(Visitor visitor) {
    visitor.visitBook(this);
  }

  @override
  double getPrice() => _price;

  String get title => _title;
  String get author => _author;
}

// Concrete Element 2: Fruit
class Fruit extends Item {
  final String _name;
  final double _price;
  final double _weight;

  Fruit(this._name, this._price, this._weight);

  @override
  void accept(Visitor visitor) {
    visitor.visitFruit(this);
  }
}
```

```
@override
double getPrice() => _price;

String get name => _name;
double get weight => _weight;
}

// Concrete Element 3: Electronic
class Electronic extends Item {
  final String _model;
  final String _brand;
  final double _price;

  Electronic(this._brand, this._model, this._price);

  @override
  void accept(Visitor visitor) {
    visitor.visitElectronic(this);
  }

  @override
  double getPrice() => _price;

  String get model => _model;
  String get brand => _brand;
}

// Concrete Visitor 1: Price Calculator
class PriceCalculator implements Visitor {
  double _totalPrice = 0;

  @override
  void visitBook(Book book) {
    _totalPrice += book.getPrice();
    print('Book: ${book.title} - \${book.getPrice()}');
  }

  @override
  void visitFruit(Fruit fruit) {
    _totalPrice += fruit.getPrice();
    print('Fruit: ${fruit.name} - \${fruit.getPrice()}');
  }

  @override
  void visitElectronic(Electronic electronic) {
    _totalPrice += electronic.getPrice();
    print('Electronic: ${electronic.brand} ${electronic.model} - \${electronic.getPrice()}');
  }

  double get totalPrice => _totalPrice;
}

// Concrete Visitor 2: Description Generator
```

```
class DescriptionGenerator implements Visitor {
  @override
  void visitBook(Book book) {
    print('Book: "${book.title}" by ${book.author} - A great read!');
  }

  @override
  void visitFruit(Fruit fruit) {
    print('Fruit: ${fruit.name} (${fruit.weight}kg) - Fresh and healthy!');
  }

  @override
  void visitElectronic(Electronic electronic) {
    print('Electronic: ${electronic.brand} ${electronic.model} - High-tech gadget!');
  }
}

// Concrete Visitor 3: Tax Calculator
class TaxCalculator implements Visitor {
  double _totalTax = 0;

  @override
  void visitBook(Book book) {
    final tax = book.getPrice() * 0.05; // 5% tax on books
    _totalTax += tax;
    print('Book tax: \${tax}');
  }

  @override
  void visitFruit(Fruit fruit) {
    final tax = fruit.getPrice() * 0.02; // 2% tax on food
    _totalTax += tax;
    print('Fruit tax: \${tax}');
  }

  @override
  void visitElectronic(Electronic electronic) {
    final tax = electronic.getPrice() * 0.10; // 10% tax on electronics
    _totalTax += tax;
    print('Electronic tax: \${tax}');
  }

  double get totalTax => _totalTax;
}

// Object Structure - manages the elements
class ShoppingCart {
  final List<Item> _items = [];

  void addItem(Item item) {
    _items.add(item);
  }
}
```

```
void acceptVisitor(Visitor visitor) {
    for (final item in _items) {
        item.accept(visitor);
    }
}

// Example 2: File System Visitor
// Shows how Visitor can be used for file system operations

abstract class FileSystemVisitor {
    void visitFile(File file);
    void visitDirectory(Directory directory);
}

abstract class FileSystemNode {
    void accept(FileSystemVisitor visitor);
    String getName();
    int getSize();
}

class File implements FileSystemNode {
    final String _name;
    final int _size;

    File(this._name, this._size);

    @override
    void accept(FileSystemVisitor visitor) {
        visitor.visitFile(this);
    }

    @override
    String getName() => _name;

    @override
    int getSize() => _size;
}

class Directory implements FileSystemNode {
    final String _name;
    final List<FileSystemNode> _children = [];

    Directory(this._name);

    void addChild(FileSystemNode child) {
        _children.add(child);
    }

    @override
    void accept(FileSystemVisitor visitor) {
        visitor.visitDirectory(this);
    }
}
```

```

    @override
    String getName() => _name;

    @override
    int getSize() {
        return _children.fold(0, (total, child) => total + child.getSize());
    }
}

class SizeCalculator implements FileSystemVisitor {
    int _totalSize = 0;

    @override
    void visitFile(File file) {
        _totalSize += file.getSize();
        print('File: ${file.getName()} - ${file.getSize()} bytes');
    }

    @override
    void visitDirectory(Directory directory) {
        print('Directory: ${directory.getName()} - ${directory.getSize()}
bytes');
        _totalSize += directory.getSize();
    }

    int get totalSize => _totalSize;
}

class XMLExporter implements FileSystemVisitor {
    @override
    void visitFile(File file) {
        print('    <file name="${file.getName()}" size="${file.getSize()}" />');
    }

    @override
    void visitDirectory(Directory directory) {
        print('    <directory name="${directory.getName()}"
size="${directory.getSize()}">');
    }
}

// Example 3: Employee Salary Calculator
// Shows how Visitor can be used for salary calculations

abstract class SalaryVisitor {
    void visitEmployee(Employee employee);
    void visitManager(Manager manager);
    void visitDeveloper(Developer developer);
    void visitDesigner(Designer designer);
}

abstract class Employee {
    void accept(SalaryVisitor visitor);
    String getName();
}

```

```
double getBaseSalary();
}

class Manager extends Employee {
  final String _name;
  final double _baseSalary;
  final double _bonus;

  Manager(this._name, this._baseSalary, this._bonus);

  @override
  void accept(SalaryVisitor visitor) {
    visitor.visitManager(this);
  }

  @override
  String getName() => _name;

  @override
  double getBaseSalary() => _baseSalary;

  double get bonus => _bonus;
}

class Developer extends Employee {
  final String _name;
  final double _baseSalary;
  final int _overtimeHours;

  Developer(this._name, this._baseSalary, this._overtimeHours);

  @override
  void accept(SalaryVisitor visitor) {
    visitor.visitDeveloper(this);
  }

  @override
  String getName() => _name;

  @override
  double getBaseSalary() => _baseSalary;

  int get overtimeHours => _overtimeHours;
}

class Designer extends Employee {
  final String _name;
  final double _baseSalary;
  final int _projectsCompleted;

  Designer(this._name, this._baseSalary, this._projectsCompleted);

  @override
  void accept(SalaryVisitor visitor) {
```

```

        visitor.visitDesigner(this);
    }

    @override
    String getName() => _name;

    @override
    double getBaseSalary() => _baseSalary;

    int get projectsCompleted => _projectsCompleted;
}

class BaseEmployee extends Employee {
    final String _name;
    final double _baseSalary;

    BaseEmployee(this._name, this._baseSalary);

    @override
    void accept(SalaryVisitor visitor) {
        visitor.visitEmployee(this);
    }

    @override
    String getName() => _name;

    @override
    double getBaseSalary() => _baseSalary;
}

class SalaryCalculator implements SalaryVisitor {
    double _totalSalary = 0;

    @override
    void visitEmployee(Employee employee) {
        final salary = employee.getBaseSalary();
        _totalSalary += salary;
        print('Employee: ${employee.getName()} - \$$salary');
    }

    @override
    void visitManager(Manager manager) {
        final salary = manager.getBaseSalary() + manager.bonus;
        _totalSalary += salary;
        print('Manager: ${manager.getName()} - \$$salary (base: \$$manager.getBaseSalary() + bonus: \$$manager.bonus)');
    }

    @override
    void visitDeveloper(Developer developer) {
        final overtimePay = developer.overtimeHours * 50.0;
        final salary = developer.getBaseSalary() + overtimePay;
        _totalSalary += salary;
        print('Developer: ${developer.getName()} - \$$salary (base:

```

```
\${developer.getBaseSalary()} + overtime: \${overtimePay}');
}

@override
void visitDesigner(Designer designer) {
  final projectBonus = designer.projectsCompleted * 500.0;
  final salary = designer.getBaseSalary() + projectBonus;
  _totalSalary += salary;
  print('Designer: ${designer.getName()} - \${salary} (base:
\${developer.getBaseSalary()} + project bonus: \${projectBonus}');
}

double get totalSalary => _totalSalary;
}

// Demo code
void main() {
  print('=== Visitor Pattern Demo ===\n');

  // Example 1: Shopping Cart
  print('--- Shopping Cart Example ---');

  final cart = ShoppingCart();
  cart.addItem(Book('Design Patterns', 'Erich Gamma', 49.99));
  cart.addItem(Fruit('Apple', 1.99, 0.5));
  cart.addItem(Electronic('Apple', 'iPhone 15', 999.99));

  print('\nCalculating total price:');
  final priceCalc = PriceCalculator();
  cart.acceptVisitor(priceCalc);
  print('Total: \${priceCalc.totalPrice}');

  print('\nGenerating descriptions:');
  final descGen = DescriptionGenerator();
  cart.acceptVisitor(descGen);

  print('\nCalculating tax:');
  final taxCalc = TaxCalculator();
  cart.acceptVisitor(taxCalc);
  print('Total tax: \${taxCalc.totalTax}');

  // Example 2: File System
  print('\n--- File System Example ---');

  final root = Directory('root');
  final file1 = File('document.txt', 1024);
  final file2 = File('image.jpg', 2048);
  final subdir = Directory('subdir');
  final file3 = File('data.csv', 512);

  subdir.addChild(file3);
  root.addChild(file1);
  root.addChild(file2);
  root.addChild(subdir);
}
```

```
print('\nCalculating sizes:');
final sizeCalc = SizeCalculator();
root.accept(sizeCalc);
print('Total size: ${sizeCalc.totalSize} bytes');

print('\nExporting to XML:');
print('<filesystem>');
final xmlExporter = XMLExporter();
root.accept(xmlExporter);
print('</filesystem>');

// Example 3: Employee Salary
print('\n--- Employee Salary Example ---');

final employees = <Employee>[
  Manager('Alice', 80000, 20000),
  Developer('Bob', 70000, 10),
  Designer('Charlie', 65000, 5)
];

print('\nCalculating total salary:');
final salaryCalc = SalaryCalculator();
for (final emp in employees) {
  emp.accept(salaryCalc);
}
print('Total salary: \${salaryCalc.totalSalary}');
}
```